



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE INGENIERÍA

ÁREA DE CIENCIAS DE LA COMPUTACIÓN

ERIK ÁNGEL DÍAZ DE LEÓN LÓPEZ
EDSON IVÁN HERNÁNDEZ SANDOVAL
ENRIQUE NAVOA PÉREZ
SERGIO FRANCISCO ZARATE CASTAÑEDA

REPORTE DE PROYECTO: UNO
REPOSITORIO: Interfaces-SalvaChad-Uno

INTERFACES GRÁFICAS CON APLICACIONES
M.I ALBERTO SALVADOR NUÑEZ VARELA

SEMESTRE 2026-2027/I

11 DE OCTUBRE DEL 2026

INTRODUCCIÓN/DESCRIPCIÓN GENERAL DEL PROYECTO

UnoGame es una recreación del juego de cartas UNO desarrollada en C# con Windows Forms para la materia de Interfaces Gráficas con Aplicaciones. El proyecto comenzó el 21 de septiembre de 2026 y se desarrolló de forma colaborativa entre cuatro integrantes, con Git y GitHub para el control de versiones y la organización del trabajo mediante milestones e issues.

La primera versión, entregada el 11 de octubre, permitía que cuatro jugadores disputaran una partida alternando turnos en una misma computadora, con una API en Python (FastAPI) y una base de datos MySQL para guardar el historial. Esta actualización lleva el proyecto al siguiente nivel: cuatro jugadores pueden jugar al mismo tiempo, cada uno en su propia computadora, con una arquitectura cliente-servidor sobre sockets TCP.

En el nuevo modelo, un servidor de juego (UnoServer) conserva el estado oficial de la partida, valida cada acción y envía a cada jugador su propia vista de la mesa. Los clientes (UnoGame) solo capturan las acciones del jugador y dibujan lo que el servidor les indica. Uno de los jugadores actúa como anfitrión: en su equipo corren MySQL, la API y el servidor, y los demás se conectan a su dirección IP.

Qué se conserva: las reglas completas del UNO, la interfaz de la mesa, las cartas dibujadas por código (GDI+), las animaciones, los sonidos y la música, el registro en MySQL mediante API REST y la ventana de historial.

Qué cambia:

• La lógica del juego (mazo, turnos, reglas, UNO y fin de partida) pasa de la ventana del juego al servidor.
• Cada jugador ve su mano completa; de los demás, solo cuántas cartas tienen.
• Hay dos pantallas nuevas: la conexión al servidor y la sala de espera.
• El servidor es quien registra la partida en la base de datos, una sola vez por partida.
• Se definió un protocolo propio de mensajes JSON, documentado en Protocolo.md.

Con esto se aplican conceptos adicionales de la materia y de la carrera: programación con sockets, hilos y sincronización, serialización JSON, separación de responsabilidades entre cliente y servidor, y pruebas sin interfaz mediante jugadores automáticos.

El desarrollo fue organizado y distribuido entre los 4 integrantes del equipo, mediante la creación de issues en el repositorio de GitHub, estos issues corresponde a un milestone designado, Lógica del Juego, Interfaz, Recurso e Integración con la base de datos a través de la API, Base y contrato, Servidor, Cliente, Historial e infraestructura y Red y Pruebas.

Apí Y BASE DE DATOS.

El proyecto esta estructurado en 3 capaz, el programa principal UnoGame.cs, unoApi.py y la base de datos en MySQL, el programa .cs y la api se comunican por medio de sentencias en formato Json y HTTP, y de la api a la base de datos se comunican por medio de sentencias sql, es decir:

- **UnoGame**: la interfaz y la lógica del juego. No sabe nada de SQL ni de MySQL.
- **unoApi.py**: un servidor intermedio. Recibe peticiones HTTP, las traduce a consultas SQL, y devuelve JSON.
- **MySQL**: solo almacena datos en tablas. No sabe que existe C# ni Python más allá de que alguien se conecta con un usuario/contraseña.

Optamos por este diseño por que, en un caso hipotético donde se cambie la base de datos por otra, o que se cambie el programa o se reescribe por algún otro lenguaje, solo seria necesario modificar la Api, esto se conoce como el patrón cliente-servidor con una API REST en medio.

unoAPi.py

- **fastapi** — el framework que define los *endpoints* (las rutas HTTP: /partidas, /historial, etc.) y convierte automáticamente los datos a/desde JSON.
- **pydantic** (viene con FastAPI) — valida que el JSON que llega tenga la forma correcta, mediante las clases NuevaPartida, NuevoMovimiento, Ganador.
- **sqlalchemy** — el motor que arma y ejecuta las consultas SQL (create_engine, text(...)).
- **pymysql** — el driver que realmente habla el protocolo de MySQL; SQLAlchemy lo usa por debajo.
- **uvicorn** — el servidor que ejecuta la API y la deja escuchando en http://127.0.0.1:5000.

En C# UnoGame, el método RegistrarMovimiento, registra una jugada y la convierte en formato Json de la siguiente manera:

```
var body = new { numero_turno = ..., nombre_jugador = ..., accion = ..., descripcion_carta = ... };
string json = JsonSerializer.Serialize(body);
```

Esta sentencia crea el formato Json como este:

```
{"numero_turno": 3, "nombre_jugador": "Ana", "accion": "Jugar", "descripcion_carta": "Rojo 7"}
```

Este Json se manda por HTTP de la siguiente manera:

```
await cliente.PostAsync($"{BaseUrl}/partidas/{partidaId}/movimientos", contenido);
```

En unoApi.py, el método de NuevoMovimiento recibe este json y lo gurda en MySQL

```
conn.execute(text("""INSERT INTO movimientos (...) VALUES (:partida_id, :numero_turno, ...)"""), {...})
```

La estructura de la API y de la base de datos no cambió; cambia quién las usa: ahora el registro lo hace UnoServer y el historial lo consultan los clientes desde cualquier equipo.

Método y ruta	Función	Quién la usa
GET /	Comprueba que la API está activa	Pruebas
POST /partidas	Crea la partida y sus cuatro participantes	UnoServer
POST /partidas/{id}/movimientos	Registra una jugada o un robo	UnoServer
PUT /partidas/{id}/ganador	Marca al ganador	UnoServer
GET /historial	Partidas con sus jugadores y resultado	UnoGame (historial)
GET /estadisticas	Victorias y derrotas por jugador	UnoGame (historial)
GET /partidas/{id}/movimientos	Movimientos de una partida	UnoGame (historial)

La API se inicia con `uvicorn unoApi:app --host 0.0.0.0 --port 5000`; el `--host 0.0.0.0` permite que otros equipos de la red la consulten. Desde el menú de UnoGame, `ApiCliente.Host` toma la última IP usada para conectarse al servidor.

Tablas (archivo `schema.sql`):

```
CREATE TABLE Partidas (
        id INT AUTO_INCREMENT PRIMARY KEY,
        fecha DATETIME NOT NULL
);

CREATE TABLE ParticipantesPartida (
        id INT AUTO_INCREMENT PRIMARY KEY,
        partida_id INT NOT NULL,
        nombre_jugador VARCHAR(100) NOT NULL,
        gano BOOLEAN NOT NULL DEFAULT FALSE,
        FOREIGN KEY (partida_id) REFERENCES Partidas(id)
);

CREATE TABLE Movimientos (
        id INT AUTO_INCREMENT PRIMARY KEY,
        partida_id INT NOT NULL,
        numero_turno INT NOT NULL,
        nombre_jugador VARCHAR(100) NOT NULL,
        accion VARCHAR(50) NOT NULL,
        descripcion_carta VARCHAR(50),
        fecha DATETIME NOT NULL
);
```

ARQUITECTURA CLIENTE SERVIDOR.

UnoGame (x4) --TCP/JSON, puerto 11111--> UnoServer --HTTP/JSON, puerto 5000--> unoApi.py --SQL--> MySQL
clientes servidor API base de datos

Componente	Tecnología	Donde Corre	Responsabilidad
UnoGame	C#, Windows Forms, .NET Framework 4.8	PC de cada jugador	Interfaz; envía las acciones del jugador y dibuja el estado recibido
UnoServer	C#, aplicación de consola, sockets TCP	PC anfitriona	Sala de espera, reglas, turnos, difusión del estado y registro en la API
unoApi.py	Python, FastAPI, SQLAlchemy, PyMySQL	PC anfitriona	Traduce las peticiones HTTP/JSON a consultas SQL
MySQL	MySQL 8	PC anfitriona	Guarda partidas, participantes y movimientos

El servidor es la única fuente de verdad: tiene el mazo y las cuatro manos, valida cada acción y decide el resultado. Los clientes solo piden acciones y muestran lo que reciben. Ventajas: es sencillo de implementar, ningún jugador puede ver cartas ajenas ni hacer jugadas ilegales, y un error en un cliente solo lo afecta a él. Inconveniente: todo depende del anfitrión; si se desconecta o tiene mala conexión, la partida se cancela para todos.
El servidor parte del ejemplo de sockets TCP visto en la materia (Bind, Listen, Accept, Connect, Send, Receive) y lo amplía en cuatro puntos:

- Conexiones persistentes: cada jugador permanece conectado durante toda la partida, en lugar de enviar un solo mensaje y cerrar.
- Un hilo por cliente, para atender a los cuatro jugadores a la vez.
- Mensajes JSON delimitados por salto de línea, en lugar de texto plano terminado en <EOF> .
- Escucha en IPAddress.Any : no depende de una IP escrita en el código; el cliente indica a qué servidor conectarse.

PROTOCOLO DE COMUNICACIÓN

1. Un mensaje es una línea de texto JSON terminada en salto de línea. TCP es un flujo de bytes sin separaciones, por eso se necesita un delimitador; el JSON nunca contiene saltos de línea reales.
2. Codificación UTF-8 y puerto por defecto 11111.
3. Un solo tipo de mensaje (Mensaje) con campos opcionales: el campo tipo indica cuáles se usan. Los nombres van en camelCase y los campos nulos no se envían.
4. El servidor decide si una acción es válida. El cliente solo la solicita y no asume que se aplicó hasta recibir el evento y el estado correspondientes; si no era válida recibe un error y la partida no cambia.
5. Cada jugador recibe un estado personalizado: su mano completa y, de los demás, solo la cantidad de cartas.

Mensajes del cliente al servidor (5)

tipo	Campos	Cuándo se envía
unirse	nombre	Al conectarse
jugar	carta, color (solo comodines)	Al jugar una carta de la mano
robar	—	Al tomar una carta de la burra
elegirColor	color	Al elegir el color de un comodín robado
decirUno	—	Al cantar UNO con dos cartas

Mensajes del servidor al cliente (8)

tipo	Campos	Significado
bienvenida	indice	Aceptado; este es tu asiento (0 a 3)
rechazo	texto	Rechazado: nombre repetido, sala llena o partida en curso
sala	nombres, texto	Quiénes esperan (por ejemplo 2/4)
estado	estado	Foto de la mesa personalizada para cada jugador
evento	texto	Lo que acaba de pasar (por ejemplo «Ana jugó Rojo 5»)
error	texto	La acción no fue válida
fin	indice, texto	La partida terminó; indice es el asiento ganador
abortada	texto	Alguien se desconectó y se canceló la partida

El estado contiene: nombres, tuIndice, turno, sentido, colorActivo, cartaSuperior, cartasBurra, cartasDescarte, cartasPorJugador, tuMano, cartaPendiente (comodín robado cuyo color falta elegir), puedesDecirUno y terminada. No existe ningún campo con las manos de los demás, por lo que el estado no puede revelar cartas ajenas.

SERVIDOR DE JUEGO UNOSERVER.

Archivo	Función
ServidorUno.cs	Red: sockets, hilos, sala de espera, difusión de mensajes y registro en la API
JuegoUno.cs	Reglas del UNO, sin nada de red ni de pantalla
Simulacion.cs, PruebasValidacion.cs, ClienteBots.cs	Herramientas de prueba
Program.cs	Punto de entrada: servidor o modos de prueba

Conexiones y sala de espera. El servidor crea un socket TCP, lo asocia al puerto 11111 (Bind), queda escuchando (Listen) y acepta conexiones (Accept) en un ciclo. Cada conexión se atiende en su propio hilo. El primer mensaje debe ser unirse; el servidor rechaza al jugador si ya hay una partida en curso, si la sala está llena o si el nombre ya está en uso. Hay cuatro asientos (0 a 3) y la partida arranca sola cuando se ocupan los cuatro.

Concurrencia. Las acciones de todos los jugadores se procesan dentro de un lock, de modo que se aplica una a la vez y el estado de la partida nunca queda a medias. Si falla el envío a un cliente, no se afecta a los demás.

Reglas en el servidor (JuegoUno).

- Mazo de 108 cartas mezclado con Fisher-Yates y 7 cartas por jugador. Si la primera carta es Comodín +4 se devuelve al mazo; si es un Comodín normal se sortea su color.
- Una carta es jugable si es comodín, o si coincide con el color activo, con el número o con el tipo de carta especial de la carta de arriba.
- El turno avanza con (turno + sentido + 4) % 4; sumar 4 evita resultados negativos cuando el sentido es -1.
- Reversa invierte el sentido; Salta hace perder el turno al siguiente; +2 y +4 hacen comer 2 y 4 cartas al siguiente, que además pierde su turno.
- Regla de UNO: con dos cartas el jugador puede cantar UNO; si juega y le queda una sin haberlo dicho, come 2 cartas.
- Al robar, si la carta se puede jugar se juega sola; si es comodín, el servidor espera el color (elegirColor); si no se puede jugar, el jugador se la queda y pasa el turno. A los demás no se les dice cuál carta se quedó.
- Si la burra se acaba, se baraja el descarte (menos la carta de arriba) para formar una nueva.
- La partida termina cuando un jugador se queda sin cartas.

Registro en la base de datos. Al iniciar la partida, el servidor la registra en la API con los cuatro nombres; después registra cada jugada y cada robo que se queda en la mano y, al final, marca al ganador. Estas llamadas van en una cola atendida por otro hilo, en orden, para que la partida no espere a la base de datos. Si la API no está disponible, el servidor escribe un aviso y la partida continúa sin guardarse.

Desconexiones. Si alguien sale en la sala de espera, los demás reciben la sala actualizada y el asiento queda libre. Si sale a mitad de partida, los demás reciben abortada y la sala se reinicia. Al terminar una partida (fin), la sala queda libre y las conexiones antiguas se cierran poco después, de modo que se puede jugar otra sin reiniciar el servidor.

5. Cliente (UnoGame)

Flujo de ventanas: FormMenu → FormConexion → FormSalaEspera → Form1 (la mesa) → FormFinPartida. Desde el fin de la partida, «Jugar de nuevo» vuelve a FormConexion con los datos ya escritos y «Volver al menú» regresa al menú.

- **FormConexion:** el jugador escribe su nombre, la IP del servidor y el puerto (por defecto 11111). Valida que no haya campos vacíos, que la IP sea una dirección IPv4 válida y que el puerto esté entre 1 y 65535. La conexión es asíncrona para no congelar la ventana, muestra el motivo si el servidor rechaza al jugador y

recuerda la última IP usada en %AppData%\UnoGame\ultima_ip.txt.

- **FormSalaEspera:** muestra los nombres conectados y el contador (por ejemplo 2/4). Cuando llega el primer estado, abre la mesa de juego. Si llega abortada o se pierde la conexión, vuelve al menú con un aviso.
- **ClienteRed:** encapsula el socket del cliente. Un hilo escucha al servidor y deja los mensajes en una cola; se entregan a la ventana activa en el hilo de la interfaz (BeginInvoke), porque los controles de Windows Forms solo se pueden modificar desde ese hilo. Con CambiarInterfaz, la conexión pasa de una ventana a otra sin perder los mensajes que llegan durante el cambio: la ventana nueva primero se suscribe y después llama a CambiarInterfaz.
- **Mesa en red (Form1.Red.cs):** AplicarEstado redibuja toda la mesa a partir del EstadoJuego recibido. Los asientos son fijos: el jugador siempre aparece abajo y, a su izquierda, arriba y a su derecha, los asientos (yo+1)%4, (yo+2)%4 y (yo+3)%4. Las cartas jugables se resaltan con una copia de la regla (JugadaValidaVista) que sirve solo como ayuda visual; quien decide es el servidor. Un clic únicamente envía un mensaje (Jugar, Robar, ElegirColor o DecirUno) y no cambia nada en pantalla hasta que el servidor responde.
- **Eventos y errores:** los eventos del servidor se muestran en una lista (se conservan las últimas 100 líneas) y los errores en una etiqueta de estado, sin ventanas modales que bloqueen la mesa.
- **Animaciones (Form1.Animaciones.cs):** son decorativas. El estado nuevo se dibuja de inmediato y las cartas «viajan» encima, calculando la diferencia entre el estado anterior y el nuevo; por eso nunca pueden dejar la pantalla distinta a lo que dice el servidor.
- **Fin de partida:** el servidor envía fin y los cuatro jugadores ven al mismo ganador; después el cliente cierra su conexión.
- **Se reutilizan** el control CartaControl (cartas dibujadas con GDI+), los efectos de sonido (SoundPlayer), la música de fondo (NAudio) y la ventana de historial.

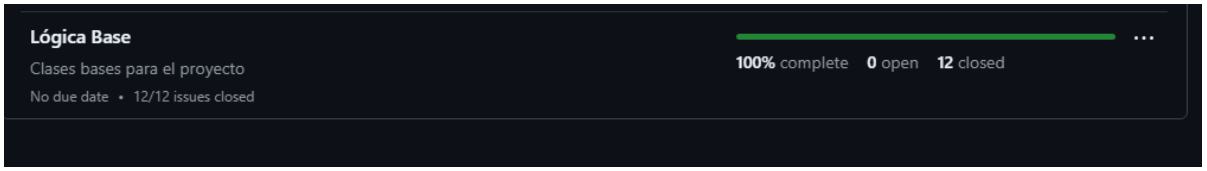
DIVISIÓN DE TAREAS

El trabajo en este proyecto se realizó de manera colaborativa, los 4 integrantes estuvieron involucrados en todo el proceso, para eso, el trabajo se organizó con ayuda de los Milestones e Issues en GitHub.
Las cuentas de GitHub pertenecen a:

ALUMNO	CUENTA EN GITHUB
Erik Angel Díaz de León López	ErikAngel
Edson Iván Hernández Sandoval	Edsonstar
Enrique Navoa Pérez	enriquenavoap
Sergio Francisco Zarate Castañeda	SergioZarateCas06

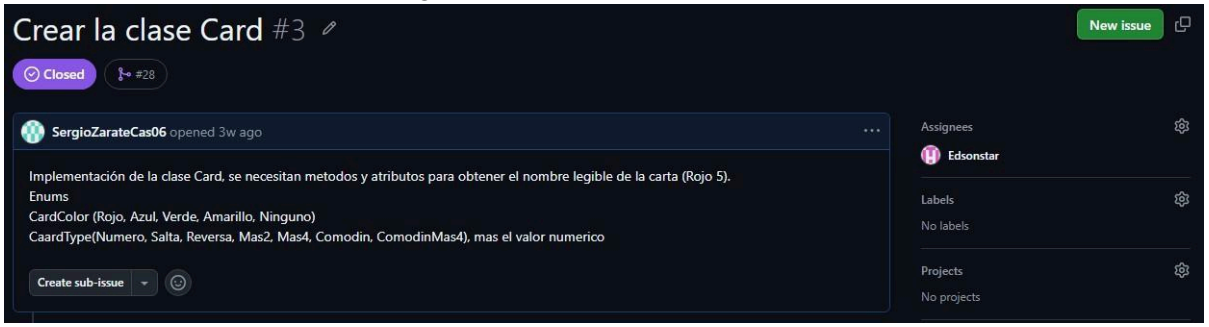
Los 4 Milestones creados son y sus respectivos Issues son:

- Lógica Base



Este Milestone se compone por 12 Issues, y se enfoca en crear el código necesario para el programa, al menos hasta la parte del funcionamiento del juego.

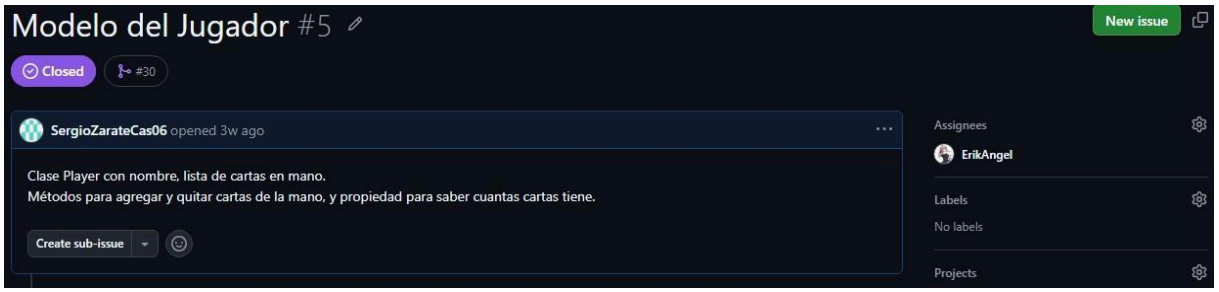
1. Crear la Clase Card. Asignado a Hernandez Sandoval



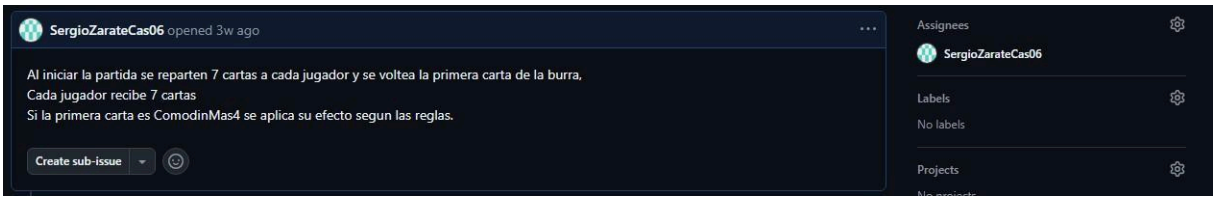
2. Generar el mazo y barajearlo. Asignado a Navoa Perez



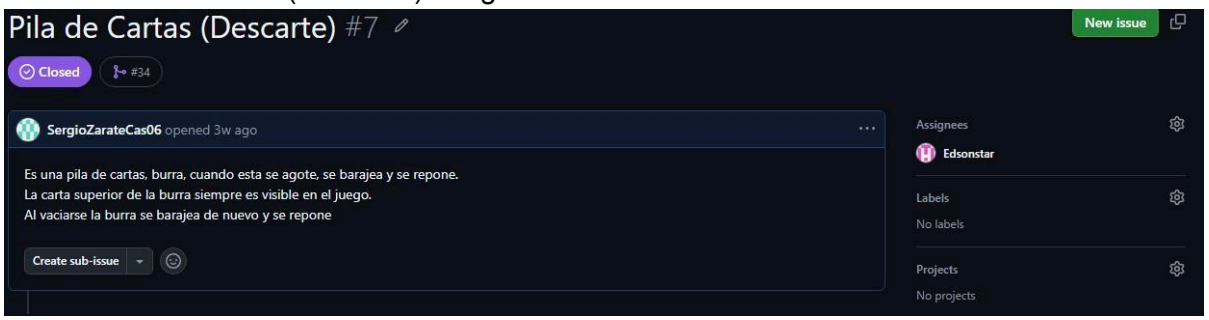
3. Modelo del jugador. Asignado a Diaz de Leon Lopez



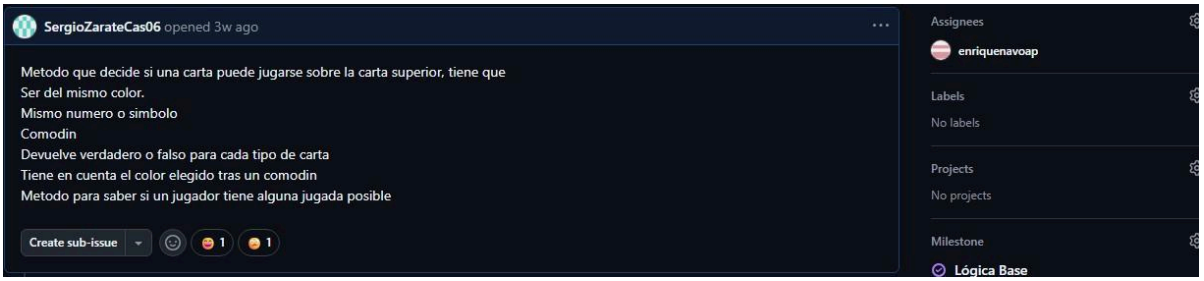
4. Repartir cartas y voltear la primera. Asignado a Zarate Castañeda



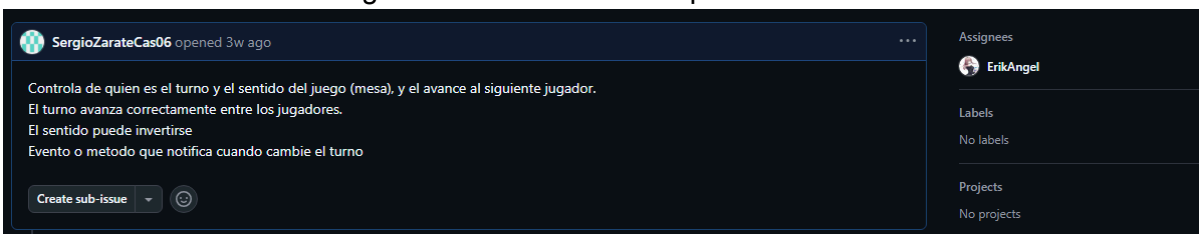
5. Pila de cartas (Descarte). Asignado a Hernandez Sandoval



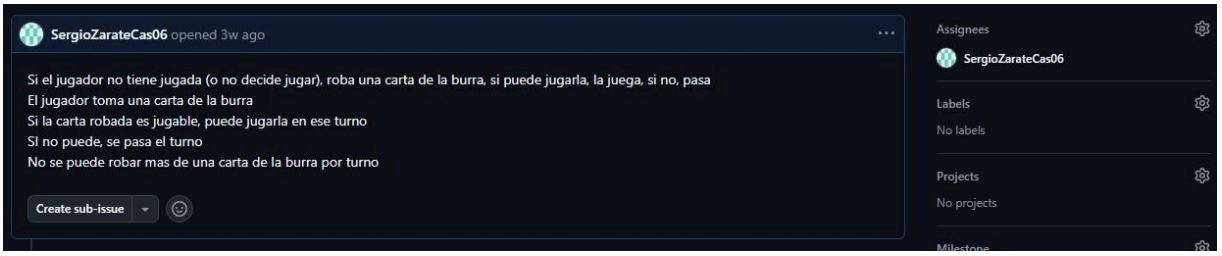
6. Validar si una jugada es legal. Asignado a Navoa Perez



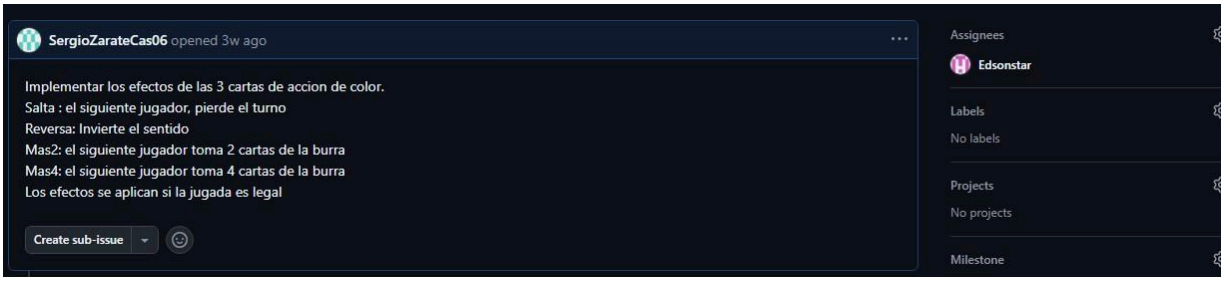
7. Motor de turnos. Asignado a Diaz de Leon Lopez



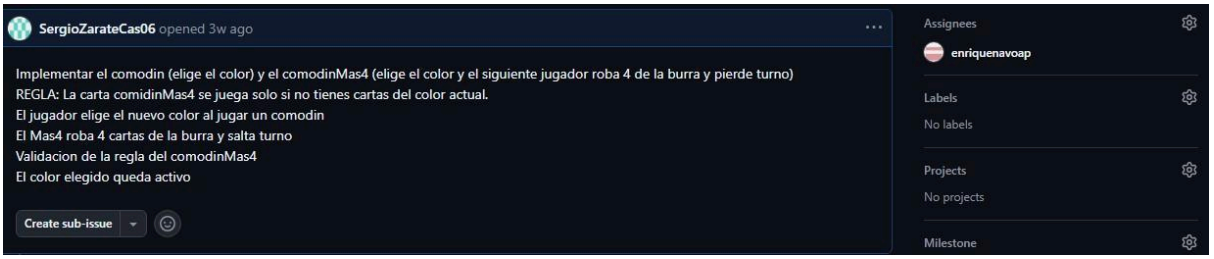
8. Robar carta y pasar turno. Asignado a Zarate Castañeda



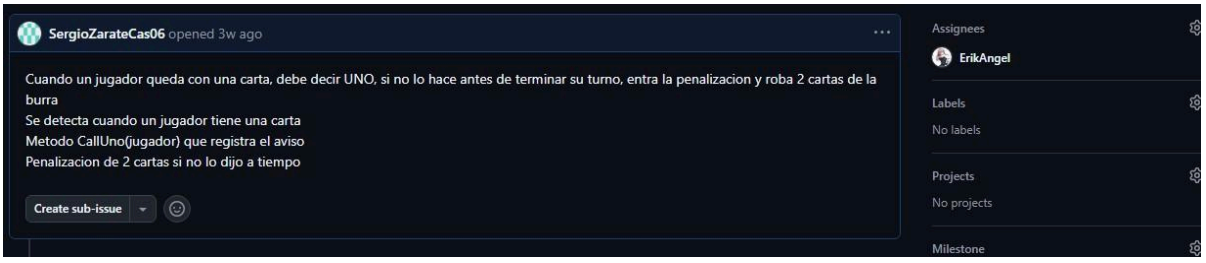
9. Cartas de acción (salta, mas2, mas4). Asignado a Hernandez Sandoval



10. Comodines y comodin mas4. Asignado a Navoa Perez



11. Regla de uno y penalización. Asignado a Diaz de Leon Lopez



12. Detectar fin de ronda y ganador. Asignado a Zarate Castañeda

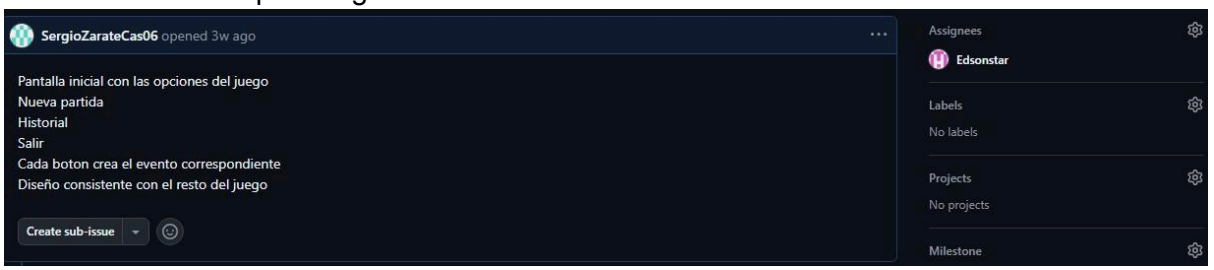


- Interfaz 10 Issues



Este Milestone se enfoca en la implementación de la interfaz gráfica pero haciendo uso de la lógica ya implementada.

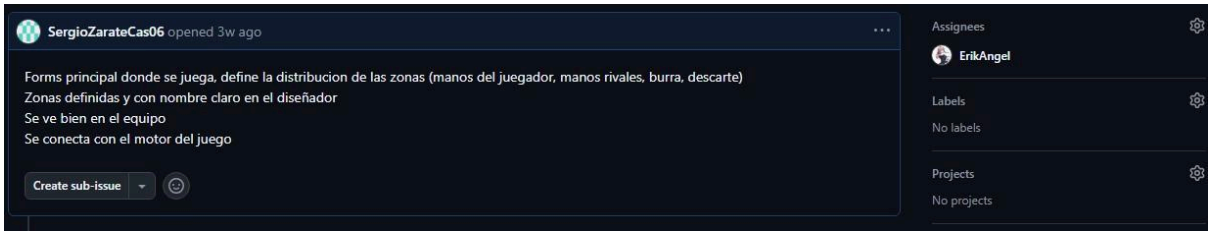
1. Menú Principal. Asignado a Hernandez Sandoval



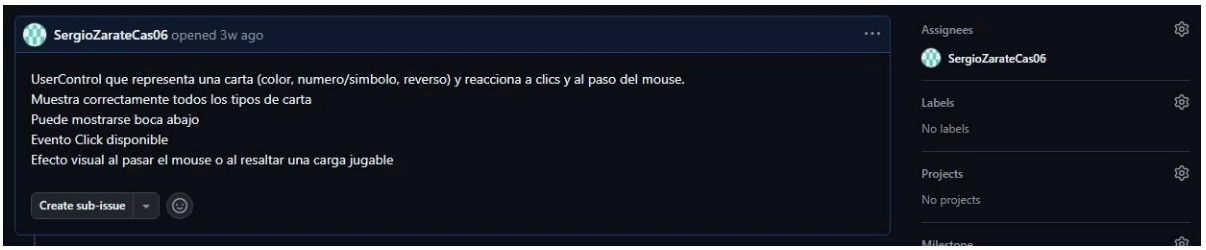
2. Pantalla de configuración de partida. Asignado a Navoa Perez



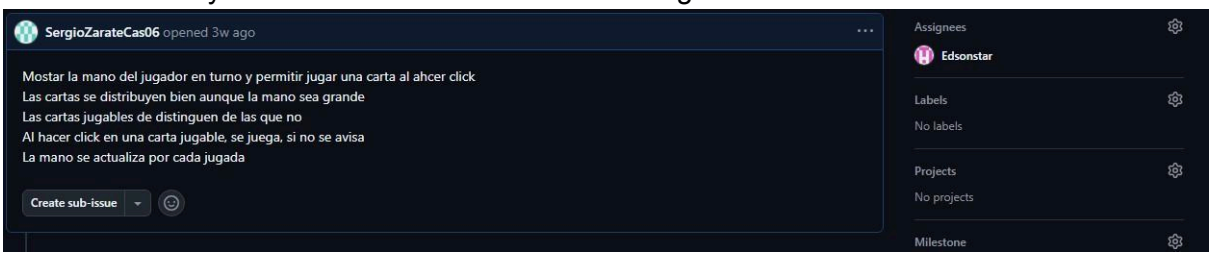
3. Forms1 de la mesa de juego. Asignado a Diaz de Leon Lopez



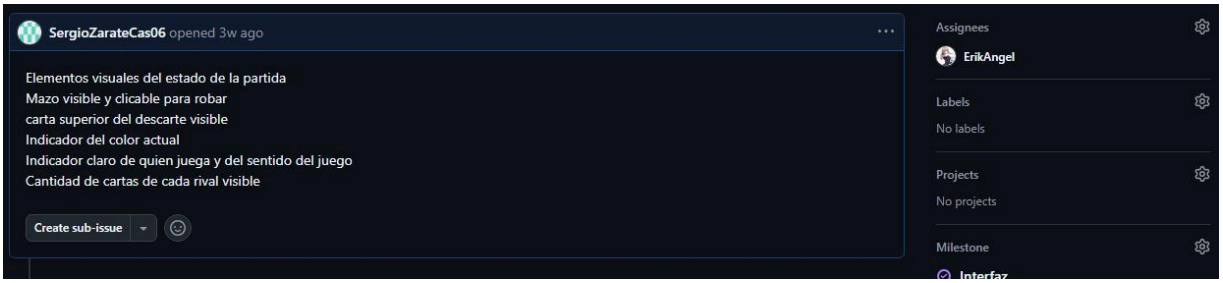
4. Control de usuario para dibujar una carta. Asignado a Zarate Castañeda.



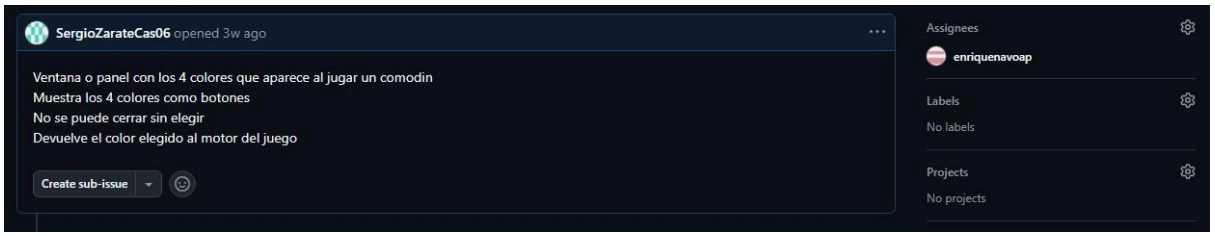
5. Mostrar y seleccionar cartas de la mano. Asignado a Hernandez Sandoval



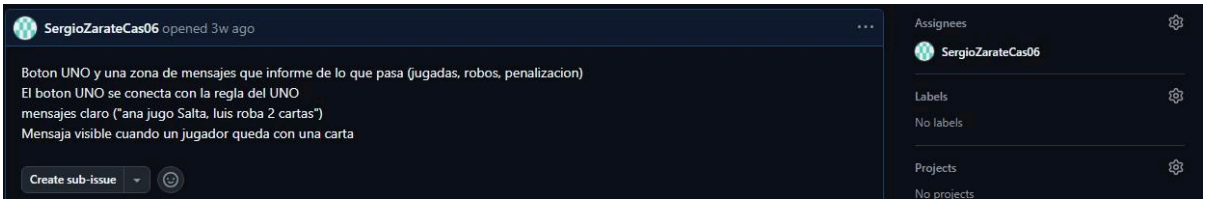
6. Mostrar mazo. descarte, color actual y mano. Asignado a Diaz de Leon Lopez



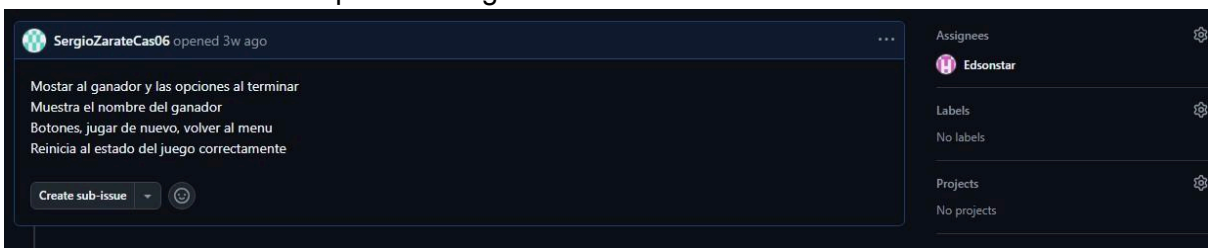
7. Diálogo para elegir color. Asignado a Navoa Perez



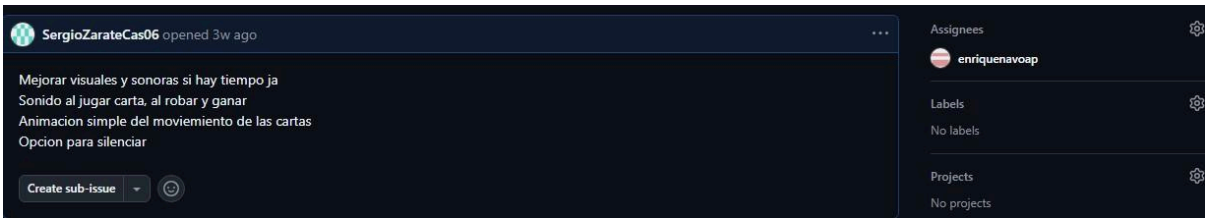
8. Botón UNO y mensaje de estado. Asignado a Zarate Castañeda



9. Pantalla de fin de partida. Asignado a Hernandez Sandoval



10. Sonidos y animaciones. Asignado a Navoa Perez

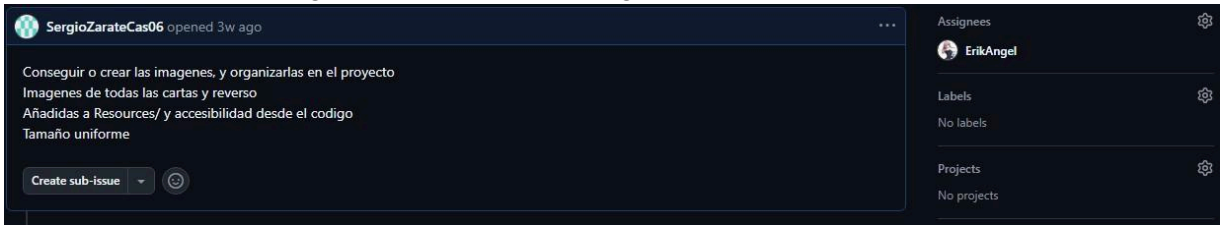


- Recursos e Integración 5 Issues

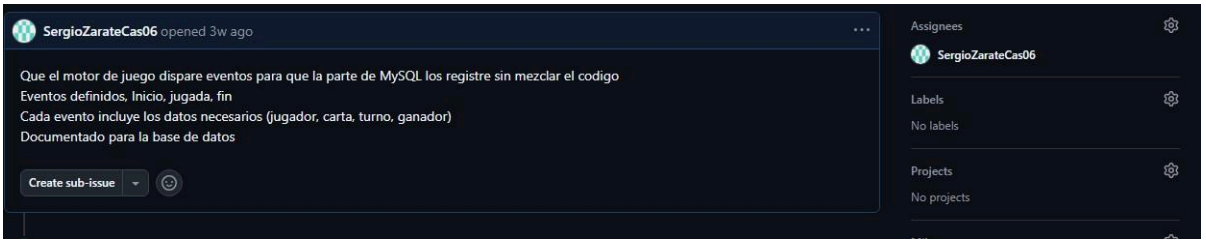


Este milestone está enfocado a integrar los recursos del programa, es decir, las imágenes de fondo, botones, cartas y sonido, además de implementar la conexión con la base de datos mediante la API.

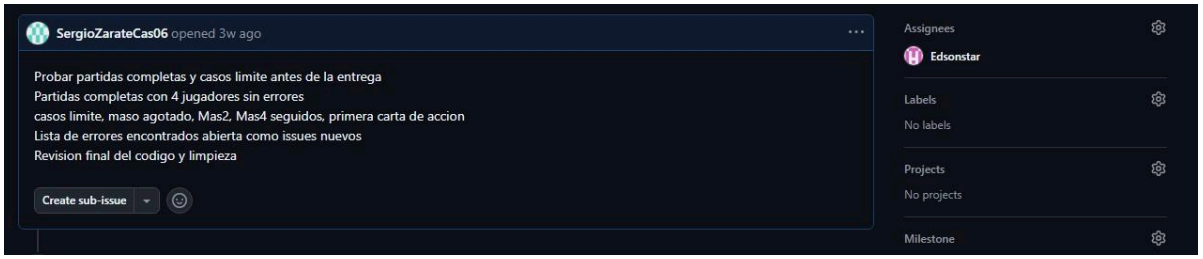
1. Reparar las imágenes de las cartas. Asignado a Diaz de Leon Lopez



2. Eventos del juego para el registro en base de datos. Asignado a Zarate Castañeda



3. Pruebas y revisión final. Asignado a Hernandez Sandoval

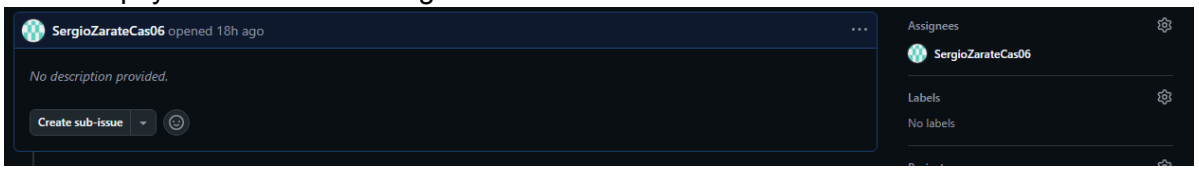


4. Música y silenciar. Asignado a Navoa Perez



Este Issue fue cerrado y en su lugar se abrió uno nuevo en Interfaz, donde sí se integraron los sonidos del juego

5. Api y Base de datos. Asignado a Zarate Castañeda

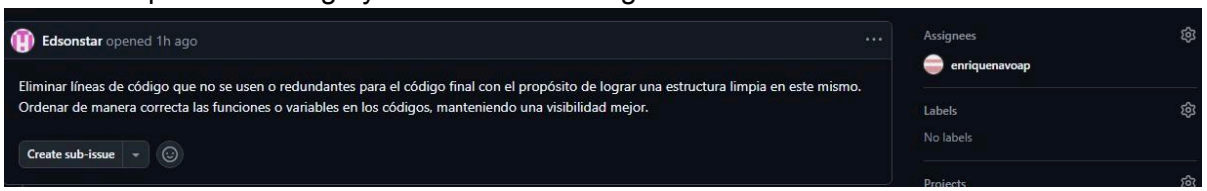


- Estructura del código 3 Issues



Este milestone se enfocó en la integración final del código, es decir, limpieza y orden, además de eliminar los cambios o eventos que quedaron obsoletos con las modificaciones que se fueron dando.

1. Limpieza de código y ordenamiento. Asignado a Navoa Perez



2. Documentación de código. Asignado a Diaz de Leon Lopez



3. Reporte. Asignado a Zarate Castañeda

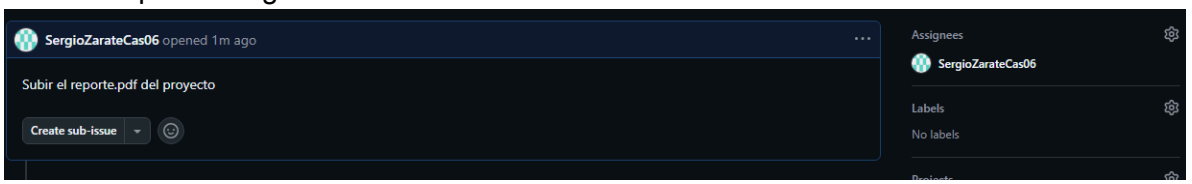
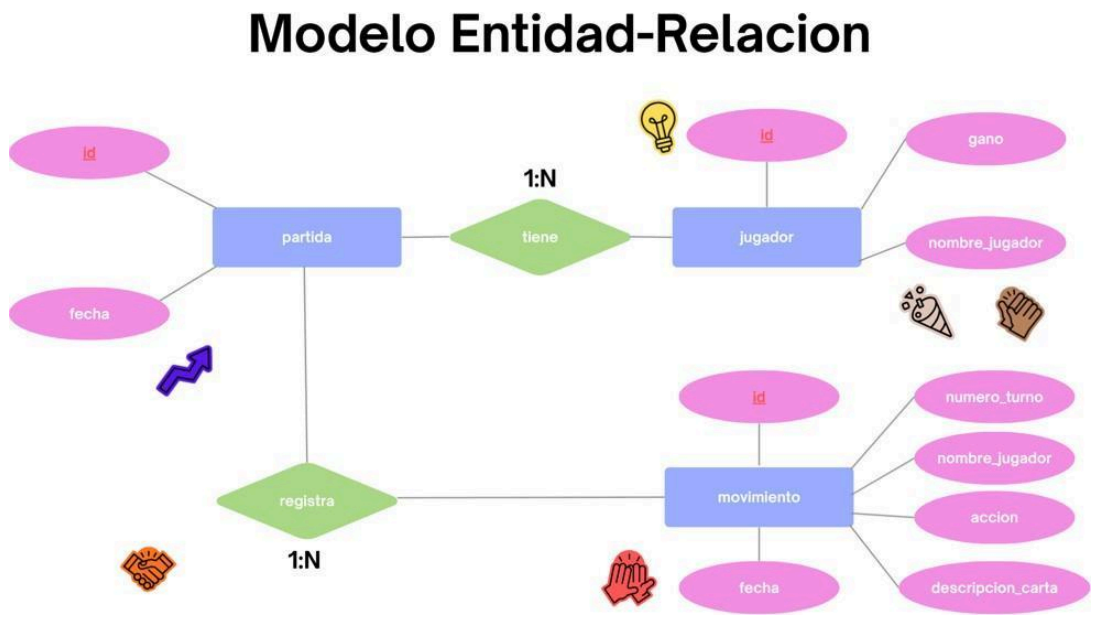
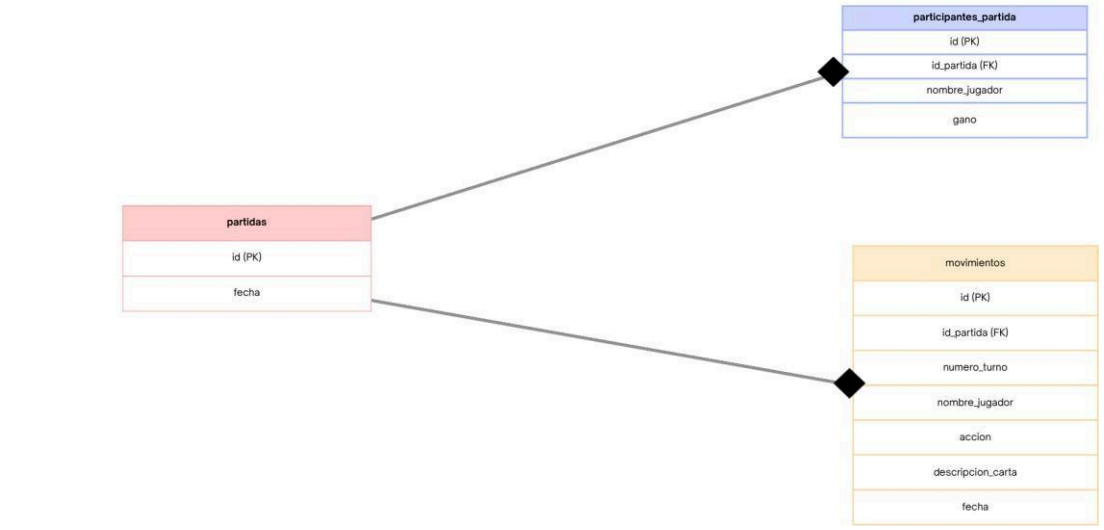


DIAGRAMA ENTIDAD-RELACIÓN





Entidades

- El recuadro partida representa la sesión de juego que se lleva a cabo
- El recuadro jugador representa quién es el jugador que gano en la sesión
- El recuadro movimiento representa cada acción que se realiza por cada jugador en cada partida

Atributos:

- id es nuestra clave primaria, indicando que este atributo identifica a cada registro
- Fecha nos dice cuando se realizó cierta acción en la partida
- Nombre_jugador y ganó nos guarda quien fue el ganador de dicha partida
- Numero_turno, acción, nombre_jugador, descripción_carta y fecha nos menciona quien realizó un movimiento, guardando su nombre y la fecha en que lo hizo

Relaciones

- La relación entre partida y jugador es de 1 a muchos, ya que una partida puede albergar a varios jugadores, pero un jugador pertenece a 1 partida en específico
- La relación entre partida y movimiento es de 1 a muchos ya que en una partida se pueden registrar muchos movimientos, pero cada movimiento ocurre exactamente en una partida

SENTENCIAS PARA CREAR LAS TABLAS EN MYSQL

```
CREATE TABLE Partidas (
	id INT AUTO_INCREMENT PRIMARY KEY,
	fecha DATETIME NOT NULL
);

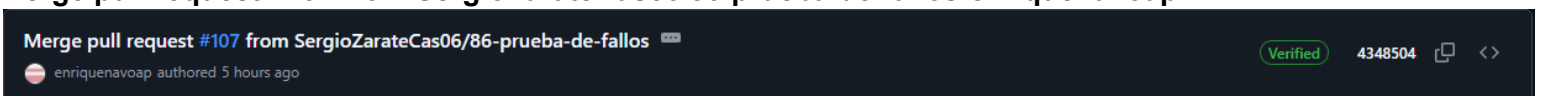
CREATE TABLE ParticipantesPartida (
	id INT AUTO_INCREMENT PRIMARY KEY,
	partida_id INT NOT NULL,
	numero_turno INT NOT NULL,
	nombre_jugador VARCHAR(100) NOT NULL,
	gano BOOLEAN NOT NULL DEFAULT FALSE,
	FOREIGN KEY (partida_id) REFERENCES Partidas(id)
);

CREATE TABLE Movimientos (
	id INT AUTO_INCREMENT PRIMARY KEY,
	partida_id INT NOT NULL,
	numero_turno INT NOT NULL,
	nombre_jugador VARCHAR(100) NOT NULL,
	accion VARCHAR(50) NOT NULL,
	descripcion_carta VARCHAR(50),
	fecha DATETIME NOT NULL
);
```

LISTADO DE COMMITS

8 de noviembre

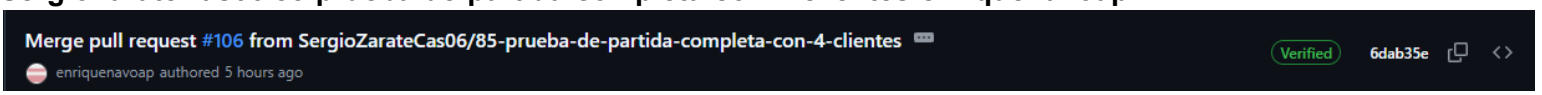
Merge pull request #107 from SergioZarateCas06/86-prueba-de-fallos-enriquenavoap



Prueba de fallos-enriquenavoap



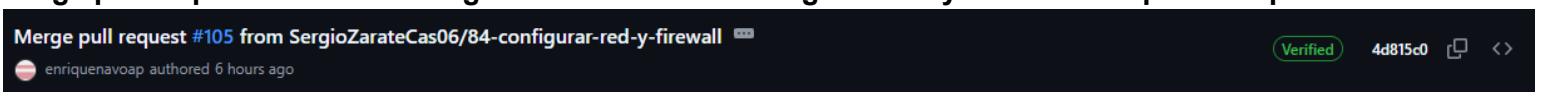
Merge pull request #106 from SergioZarateCas06/85-prueba-de-partida-completa-con-4-clientes-enriquenavoap



Prueba de partida completa-enriquenavoap



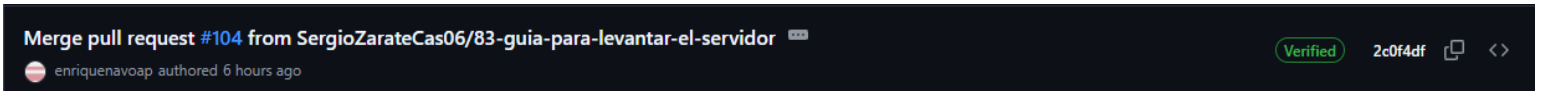
Merge pull request #105 from SergioZarateCas06/84-configurar-red-y-firewall-enriquenavoap



configurar red y firewall-enriquenavoap



Merge pull request #104 from SergioZarateCas06/83-guia-para-levantar-el-servidor-enriquenavoap



Create instrucciones.md - enriquenavoap



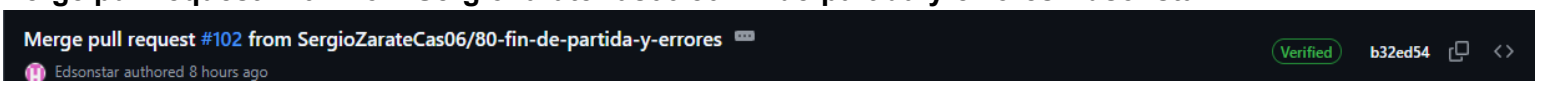
Merge pull request #103 from SergioZarateCas06/82-historial-por-red



historial por red - enriquenavoap



Merge pull request #102 from SergioZarateCas06/80-fin-de-partida-y-errores-Edsonstar

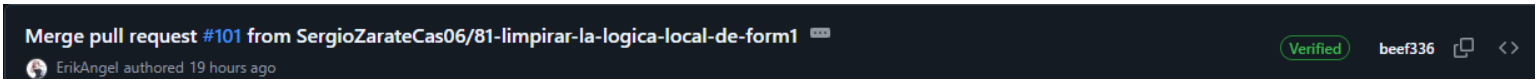


Pantalla de final de partida y problemas con conexion-Edsonstar



7 de noviembre

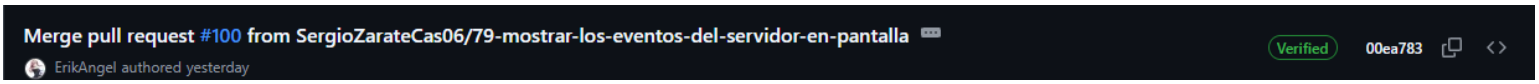
Merge pull request #101 from SergioZarateCas06/81-limpirar-la-logica-local-de-form1-ErikAngel



limpirar la logica local de form1-ErikAngel



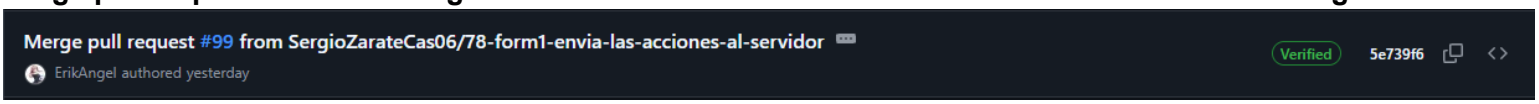
Merge pull request #100 from SergioZarateCas06/79-mostrar-los-eventos-del-servidor-en-pantalla-ErikAngel



mostrar los eventos del servidor en pantalla-ErikAngel



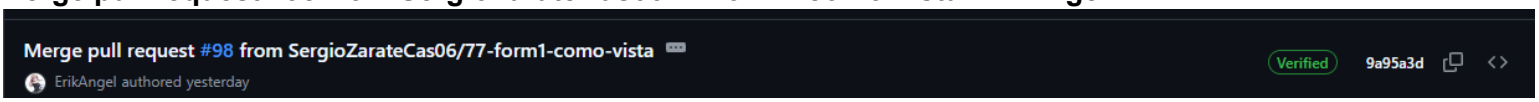
Merge pull request #99 from SergioZarateCas06/78-form1-envia-las-acciones-al-servidor-ErikAngel



Form1 envia las acciones al servidor-ErikAngel



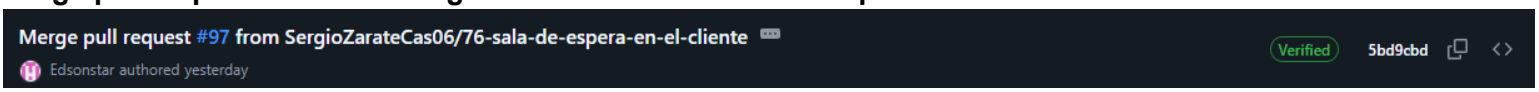
Merge pull request #98 from SergioZarateCas06/77-form1-como-vista-ErikAngel



Form1 como vista-ErikAngel



Merge pull request #97 from SergioZarateCas06/76-sala-de-espera-en-el-cliente-Edsonstar



Sala de Espera-Edsonstar



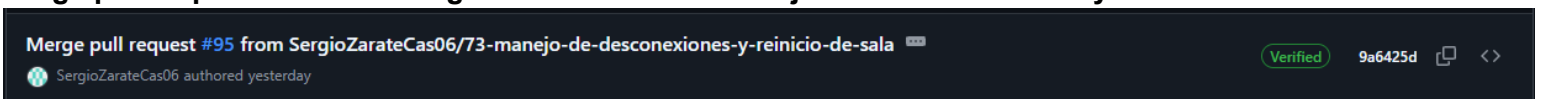
Merge pull request #96 from SergioZarateCas06/75-pantalla-de-conexion-Edsonstar



Diseño de la interfaz para la pantalla de conexion-Edsonstar



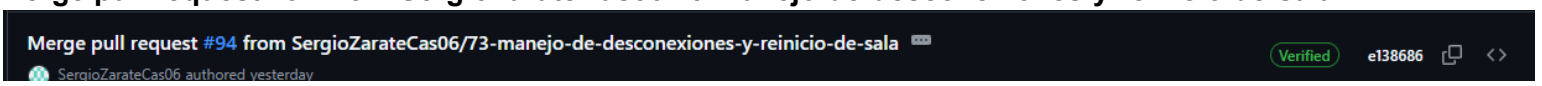
Merge pull request #95 from SergioZarateCas06/73-manejo-de-desconexiones-y-reinicio-de-sala



prueba de base de datos-SergioZarateCas06



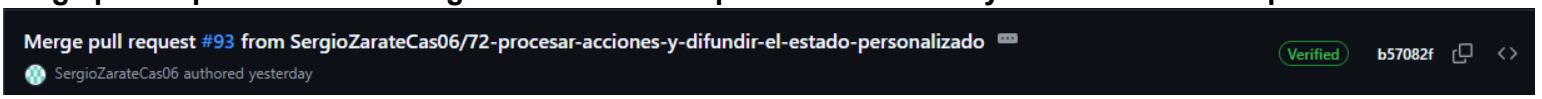
Merge pull request #94 from SergioZarateCas06/73-manejo-de-desconexiones-y-reinicio-de-sala



Prueba de pártida-SergioZarateCas06



Merge pull request #93 from SergioZarateCas06/72-procesar-acciones-y-difundir-el-estado-personalizado

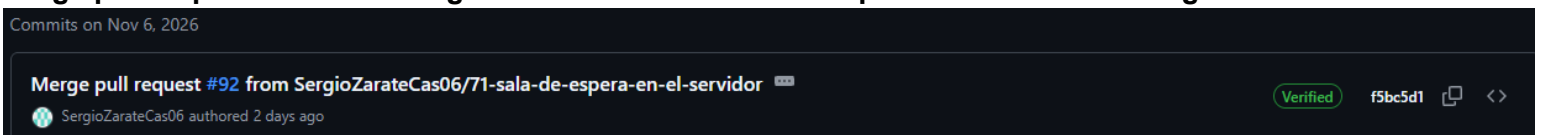


Validaciones-SergioZarateCas06



6 de noviembre

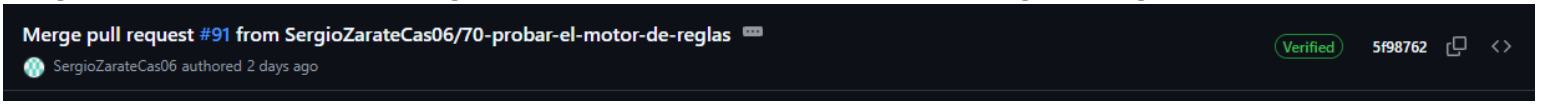
Merge pull request #92 from SergioZarateCas06/71-sala-de-espera-en-el-servidor-SergioZarateCas06



prueba de jugadores-SergioZarateCas06



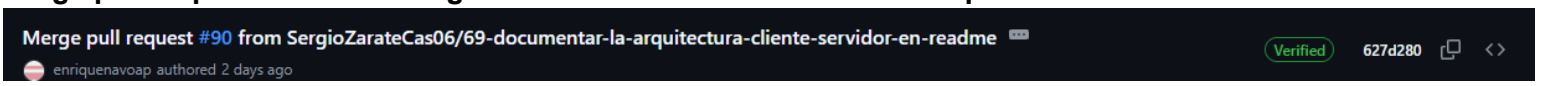
Merge pull request #91 from SergioZarateCas06/70-probar-el-motor-de-reglas-SergioZarateCas06



motor de reglas-SergioZarateCas06



Merge pull request #90 from SergioZarateCas06/69-documentar-la-arquitectura-cliente-servidor-en-readme



Update README.md-enriquenaviap



Merge pull request #89 from SergioZarateCas06/63-reporte-SergioZarateCas06

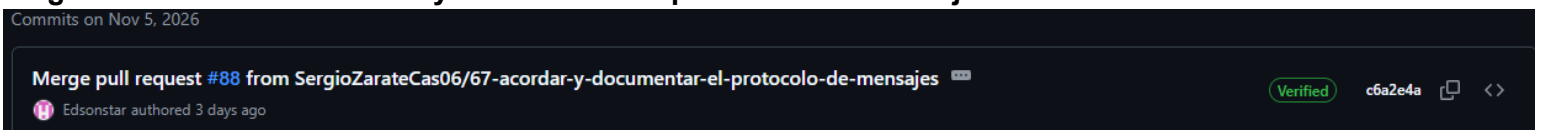


Integrar UnoServer a la solución: protocolo, servidor y cliente de red-SergioZarateCas06



5 de noviembre

Merge pull request #88 from SergioZarateCas06/67-acordar-y-documentar-el-protocolo-de-mensajes



Creación de la documentación del protocolo EdsonStar



- **Vinculacion a la BD - SergioZarateCas06**




```
- CREATE TABLE Partidas (
      id INT AUTO_INCREMENT PRIMARY KEY,
      fecha DATETIME NOT NULL
  );

- CREATE TABLE ParticipantesPartida (
      id INT AUTO_INCREMENT PRIMARY KEY,
      partida_id INT NOT NULL,
      nombre_jugador VARCHAR(100) NOT NULL,
      gano BOOLEAN NOT NULL DEFAULT FALSE,
      FOREIGN KEY (partida_id) REFERENCES Partidas(id)
  );

- CREATE TABLE Movimientos (
      id INT AUTO_INCREMENT PRIMARY KEY,
      partida_id INT NOT NULL,
      numero_turno INT NOT NULL,
      nombre_jugador VARCHAR(100) NOT NULL,
-     accion VARCHAR(50) NOT NULL,        -- "Jugar", "Robar", "ComerPenalizacion"
-     descripcion_carta VARCHAR(50),      -- ej. "Rojo 7", "Comodín +4",
```

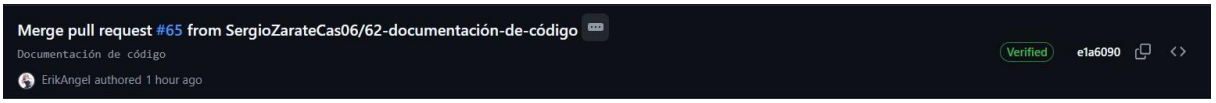
```
+ CREATE TABLE `unogameproyecto`.`Partidas` (
      id INT AUTO_INCREMENT PRIMARY KEY,
      fecha DATETIME NOT NULL
  );

+ CREATE TABLE `unogameproyecto`.`ParticipantesPartida` (
      id INT AUTO_INCREMENT PRIMARY KEY,
      partida_id INT NOT NULL,
      nombre_jugador VARCHAR(100) NOT NULL,
      gano BOOLEAN NOT NULL DEFAULT FALSE,
      FOREIGN KEY (partida_id) REFERENCES Partidas(id)
  );

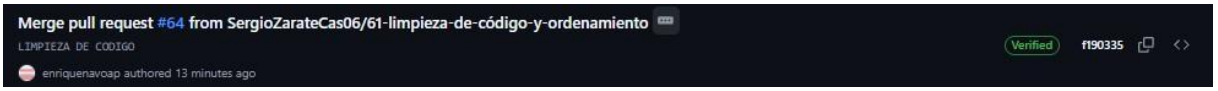
+ CREATE TABLE `unogameproyecto`.`Movimientos` (
      id INT AUTO_INCREMENT PRIMARY KEY,
      partida_id INT NOT NULL,
      numero_turno INT NOT NULL,
      nombre_jugador VARCHAR(100) NOT NULL,
+     accion VARCHAR(50) NOT NULL,

+     descripcion_carta VARCHAR(50),
```

- **Merge pull request #65 from SergioZarateCas06/62-documentación-de-código - ErikAngel**



- **Documentación de código - ErikAngel**
- **Merge pull request #64 from SergioZarateCas06/61-limpieza-de-código-y-ordenamiento - enriquenavoap**



- **LIMPIEZA DE CODIGO - enriquenavoap**



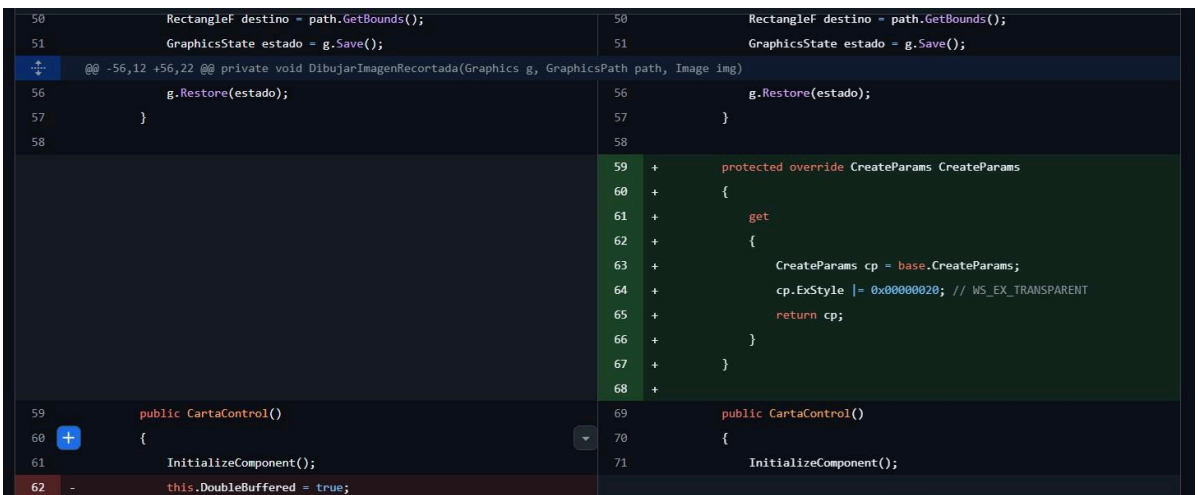
- **Actualización de README - Edsonstar**





- **Merge pull request #60 from SergioZarateCas06/27-pruebas-y-revision-final - Edsonstar**



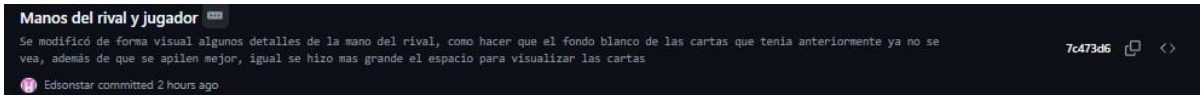


- **Arreglo de la burra - Edsonstar**




```
695
+        private void MostrarAviso(string mensaje)
+        {
+            if (IsDisposed) return;
+            bool corria = timerReparto.Enabled;
+            timerReparto.Stop();
+            MessageBox.Show(mensaje, "Burra");
+            if (corria && !IsDisposed) timerReparto.Start();
+        }
+
+        private void AlAcabarseLaBurra()
+        {
+            string mensaje;
+            if (descarte.Count > 1)
+            {
+                ReponerMazo();   //BARAJEAR BURRA
+                mensaje = "Se acabó la burra. Se barajó el descarte para formar una nueva.";
+            }
+            else
+            {
+                mensaje = "Se acabó la burra y no hay cartas en el descarte para reponerla.";
+            }
+
```

- **Manos del rival y jugador - Edsonstar**




```
            g.Restore(estado);
        }

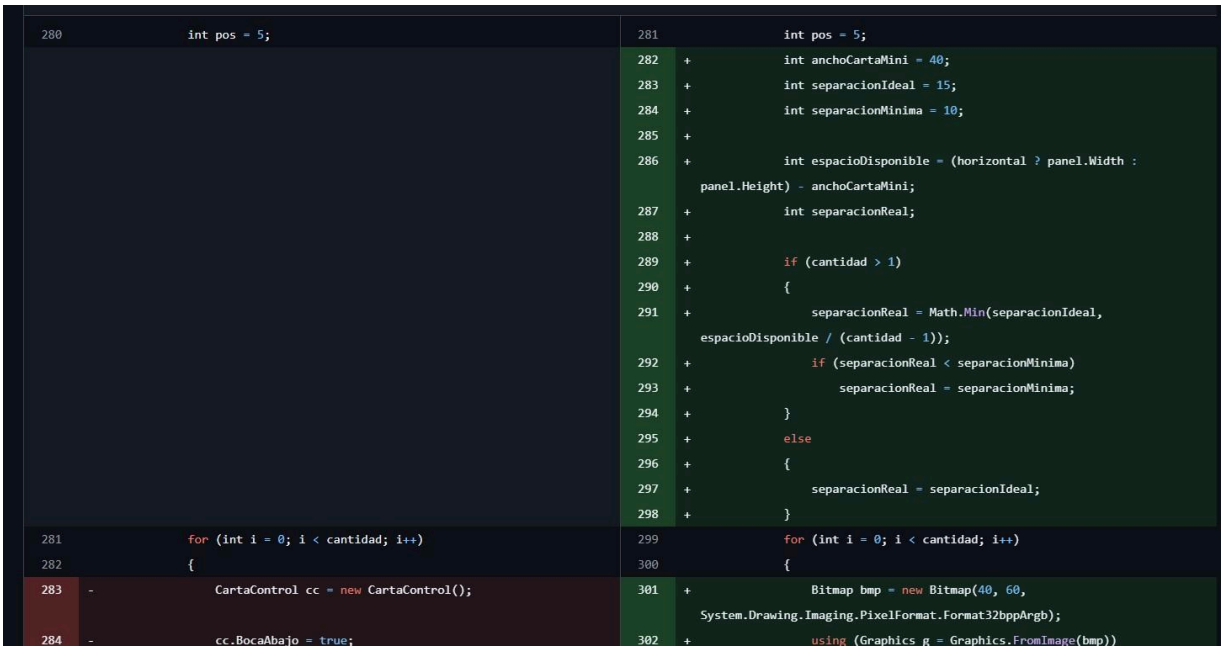
        public CartaControl()
        {
            InitializeComponent();
-           this.DoubleBuffered = true;
            this.Size = new Size(70, 100);
            this.Cursor = Cursors.Hand;

            this.MouseEnter += CartaControl_MouseEnter;
            this.MouseLeave += CartaControl_MouseLeave;
@@ -108,7 +118,6 @@ protected override void OnPaint(PaintEventArgs e)
            g.Restore(estado);
        }

+       protected override CreateParams CreateParams
+       {
+           get
+           {
+               CreateParams cp = base.CreateParams;
+               cp.ExStyle |= 0x00000020; // WS_EX_TRANSPARENT
+               return cp;
+           }
+       }
+
        public CartaControl()
        {
            InitializeComponent();

            this.Size = new Size(70, 100);
            this.Cursor = Cursors.Hand;
+           this.BackColor = Color.Transparent;

            this.MouseEnter += CartaControl_MouseEnter;
            this.MouseLeave += CartaControl_MouseLeave;
```



```
280             int pos = 5;
281             for (int i = 0; i < cantidad; i++)
282             {
283 -               CartaControl cc = new CartaControl();
284 -               cc.BocaAbajo = true;
```

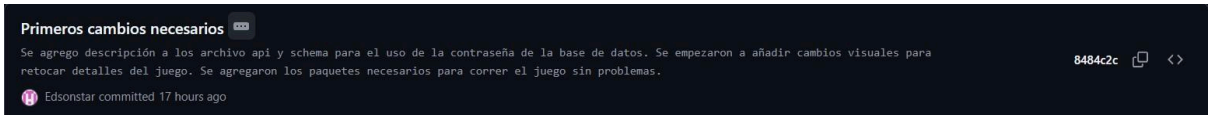
```
281             int pos = 5;
282 +           int anchoCartaMini = 40;
283 +           int separacionIdeal = 15;
284 +           int separacionMinima = 10;
285 +
286 +           int espacioDisponible = (horizontal ? panel.Width :
    panel.Height) - anchoCartaMini;
287 +           int separacionReal;
288 +
289 +           if (cantidad > 1)
290 +           {
291 +               separacionReal = Math.Min(separacionIdeal,
    espacioDisponible / (cantidad - 1));
292 +               if (separacionReal < separacionMinima)
293 +                   separacionReal = separacionMinima;
294 +           }
295 +           else
296 +           {
297 +               separacionReal = separacionIdeal;
298 +           }
299             for (int i = 0; i < cantidad; i++)
300             {
301 +               Bitmap bmp = new Bitmap(40, 60,
    System.Drawing.Imaging.PixelFormat.Format32bppArgb);
302 +               using (Graphics g = Graphics.FromImage(bmp))
```

- **Cambios visuales - Edsonstar**

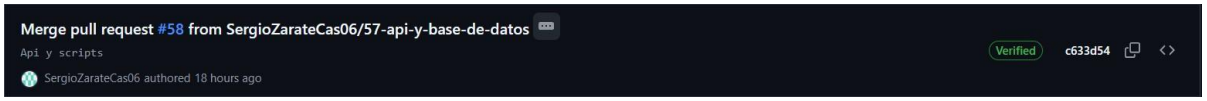


Oct 10, 2026

- **Primeros cambios necesarios - Edsonstar**



- **Merge pull request #58 from SergioZarateCas06/57-api-y-base-de-datos - SergioZarateCas06**



schema.sql +29

@@ -0,0 +1,29 @@

```
+ CREATE DATABASE IF NOT EXISTS unogameproyecto;
+ USE unogameproyecto;
+
+ CREATE TABLE Partidas (
+     id INT AUTO_INCREMENT PRIMARY KEY,
+     fecha DATETIME NOT NULL
+ );
+
+ CREATE TABLE ParticipantesPartida (
+     id INT AUTO_INCREMENT PRIMARY KEY,
+     partida_id INT NOT NULL,
+     nombre_jugador VARCHAR(100) NOT NULL,
+     gano BOOLEAN NOT NULL DEFAULT FALSE,
+     FOREIGN KEY (partida_id) REFERENCES Partidas(id)
+ );
+
+ CREATE TABLE Movimientos (
+     id INT AUTO_INCREMENT PRIMARY KEY,
+     partida_id INT NOT NULL,
+     numero_turno INT NOT NULL,
+     nombre_jugador VARCHAR(100) NOT NULL,
+     accion VARCHAR(50) NOT NULL,       -- "Jugar", "Robar",
"ComerPenalizacion"
```

unoApi.py +115

@@ -0,0 +1,115 @@

```
+ from fastapi import FastAPI
+ from sqlalchemy import create_engine, text
+ from pydantic import BaseModel
+ from typing import List, Optional
+ from datetime import datetime
+
+ app = FastAPI()
+
+ connStr =
"mysql+pymysql://uno_api:ClaveSegura2026@localhost:3306/unogameproyecto
"
+ engine = create_engine(connStr)
+
+
+ class NuevaPartida(BaseModel):
+     jugadores: List[str]
+
+ class NuevoMovimiento(BaseModel):
+     numero_turno: int
+     nombre_jugador: str
+     accion: str
+     descripcion_carta: Optional[str] = None
```

- **Api y scripts - SergioZarateCas06**

Api y scripts

El programa de la api, se conecta con una bd local, cuando uno lo corra tiene que cambiar las claves de acceso del connstr y acceda a la bd, que esta debe ser creada con las sentencias del script del archivo schema.sql

eb3acdb

SergioZarateCas06 committed 18 hours ago

- **Merge pull request #56 from SergioZarateCas06/26-eventos-del-juego-para-el-registro-en-base-de-datos - SergioZarateCas06**

Merge pull request #56 from SergioZarateCas06/26-eventos-del-juego-para-el-registro-en-base-de-datos

26 eventos del juego para el registro en base de datos

Verified bc7d62b

SergioZarateCas06 authored 19 hours ago

UnoGame/ApiCliente.cs +134

@@ -0,0 +1,134 @@

```
+ using System;
+ using System.Collections.Generic;
+ using System.Net.Http;
+ using System.Text;
+ using System.Text.Json;
+ using System.Threading.Tasks;
+
+ namespace UnoGame
+ {
+     public class ApiCliente
+     {
+         private static readonly HttpClient cliente = new HttpClient();
+         private const string BaseUrl = "http://127.0.0.1:5000";
+
+         // POST /partidas -> devuelve el partida_id creado
+         public async Task<int> CrearPartida(List<string> jugadores)
+         {
+             var body = new { jugadores = jugadores };
+             string json = JsonSerializer.Serialize(body);
+             var contenido = new StringContent(json, Encoding.UTF8,
"application/json");
+
+             HttpResponseMessage respuesta = await cliente.PostAsync($"
```

- **Merge branch ‘main’ into 26-eventos-del-juego-para-el-registro-en-base-de-datos - SergioZarateCas06**

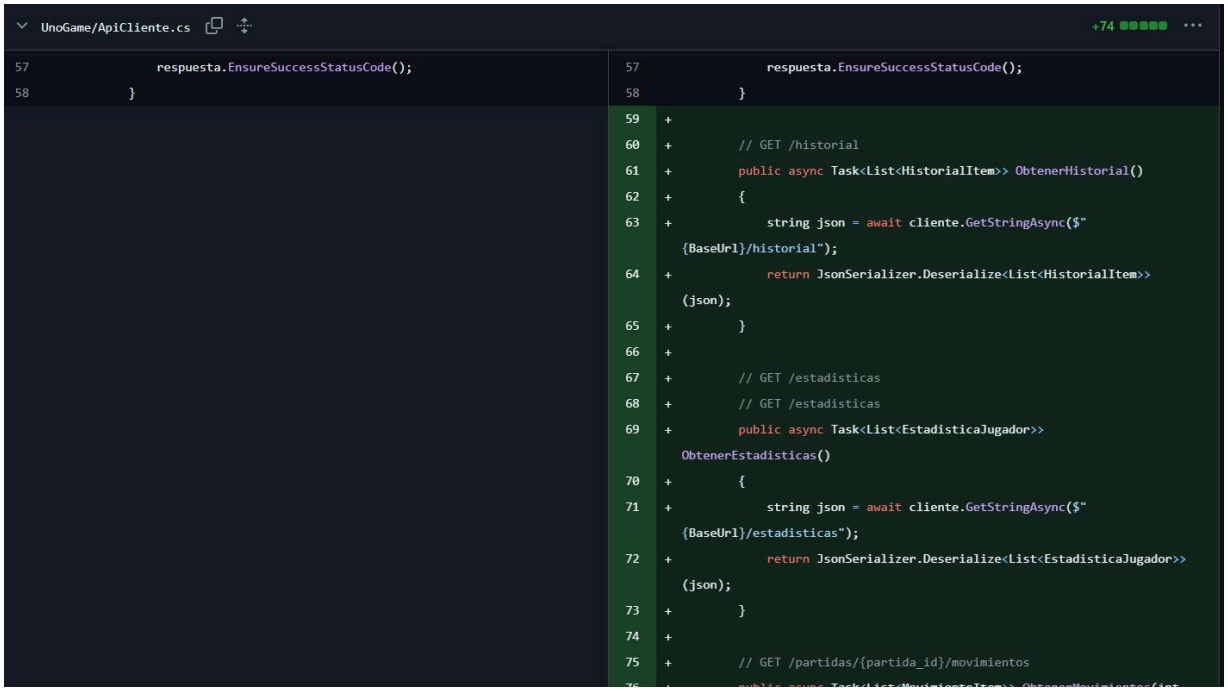
Merge branch 'main' into 26-eventos-del-juego-para-el-registro-en-base-de-datos

SergioZarateCas06 authored 19 hours ago Verified 25b3d0b

```
+         void IniciarMusica()
+         {
+             try
+             {
+                 lectorMusica = new
Mp3FileReader(Properties.Resources.musica);
+                 salidaMusica = new WaveOutEvent();
+                 salidaMusica.Init(lectorMusica);
+                 salidaMusica.Volume = 0.25f;   // 0.0 a 1.0
+
+                 // Repetir en bucle al terminar
+                 salidaMusica.PlaybackStopped += (s, e) =>
+                 {
+                     if (cerrando) return;
+                     lectorMusica.Position = 0;
+                     salidaMusica.Play();
+                 };
+
+                 salidaMusica.Play();
+             }
+             catch (Exception ex)
+             {
+                 MessageBox.Show("No se pudo iniciar la música: " +
ex.Message);
+             }
```

Conexion con la BD - SergioZarateCas06

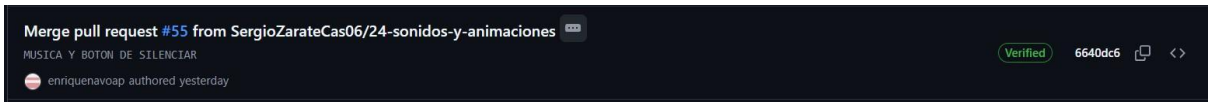




- **conexion con la base de datos - SergioZarateCas06**



- **Merge pull request #55 from SergioZarateCas06/24-sonidos-y-animaciones - enriquenavoap**



```
                salidaMusica.Init(lectorMusica);
                salidaMusica.Volume = 0.25f;   // 0.0 a 1.0
205
                // Repetir en bucle al terminar
                salidaMusica.PlaybackStopped += (s, e) =>
                {
                    if (cerrando) return;
                    lectorMusica.Position = 0;
                    salidaMusica.Play();
                };

                salidaMusica.Play();
            }
            catch (Exception ex)
            {
                MessageBox.Show("No se pudo iniciar la música: " +
ex.Message);
            }
        }

        protected override void OnFormClosed(FormClosedEventArgs e)
        {
            cerrando = true;
            if (salidaMusica != null) salidaMusica.Dispose();
            if (lectorMusica != null) lectorMusica.Dispose();
            base.OnFormClosed(e);
```

- **MUSICA Y BOTON DE SILENCIAR - enriquenavoap**

```
MUSICA Y BOTON DE SILENCIAR
AGREGUE LA MUSICA, UN BOTON DE SILENCIAR Y UN SONIDO DE ERROR
enriquenavoap committed yesterday                                  bc5579c
```

```
UnoGame/Form1.cs                                                   +67 -5
204         }
            }
            void IniciarMusica()
            {
                try
                {
                    lectorMusica = new
Mp3FileReader(Properties.Resources.musica);
                    salidaMusica = new WaveOutEvent();
                    salidaMusica.Init(lectorMusica);
                    salidaMusica.Volume = 0.25f;   // 0.0 a 1.0
205
                    // Repetir en bucle al terminar
                    salidaMusica.PlaybackStopped += (s, e) =>
                    {
                        if (cerrando) return;
                        lectorMusica.Position = 0;
                        salidaMusica.Play();
                    };

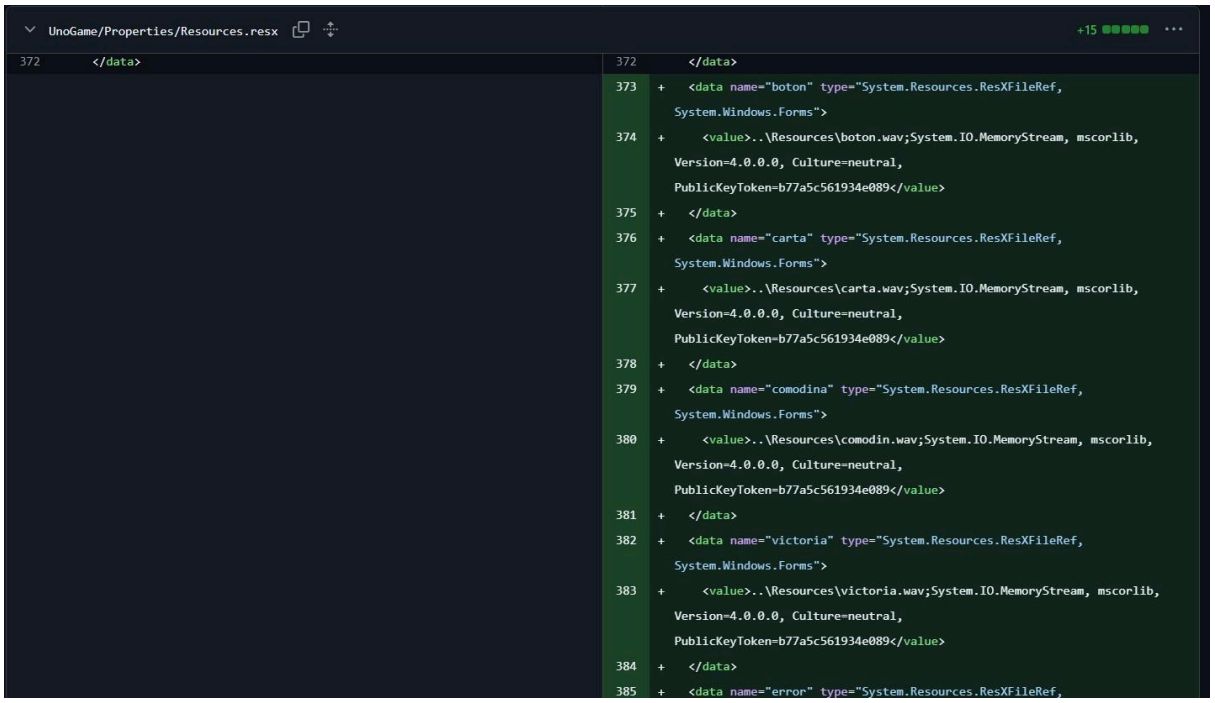
                    salidaMusica.Play();
                }
                catch (Exception ex)
                {
                    MessageBox.Show("No se pudo iniciar la música: " +
ex.Message);
```

- **Merge pull request #53 from SergioZarateCas06/24-sonidos-y-animaciones - enriquenavoap**

```
Merge pull request #53 from SergioZarateCas06/24-sonidos-y-animaciones
24sonidosyanimaciones
enriquenavoap authored 2 days ago                  Verified   829f7af
```

- **24sonidosyanimaciones - enriquenavoap**

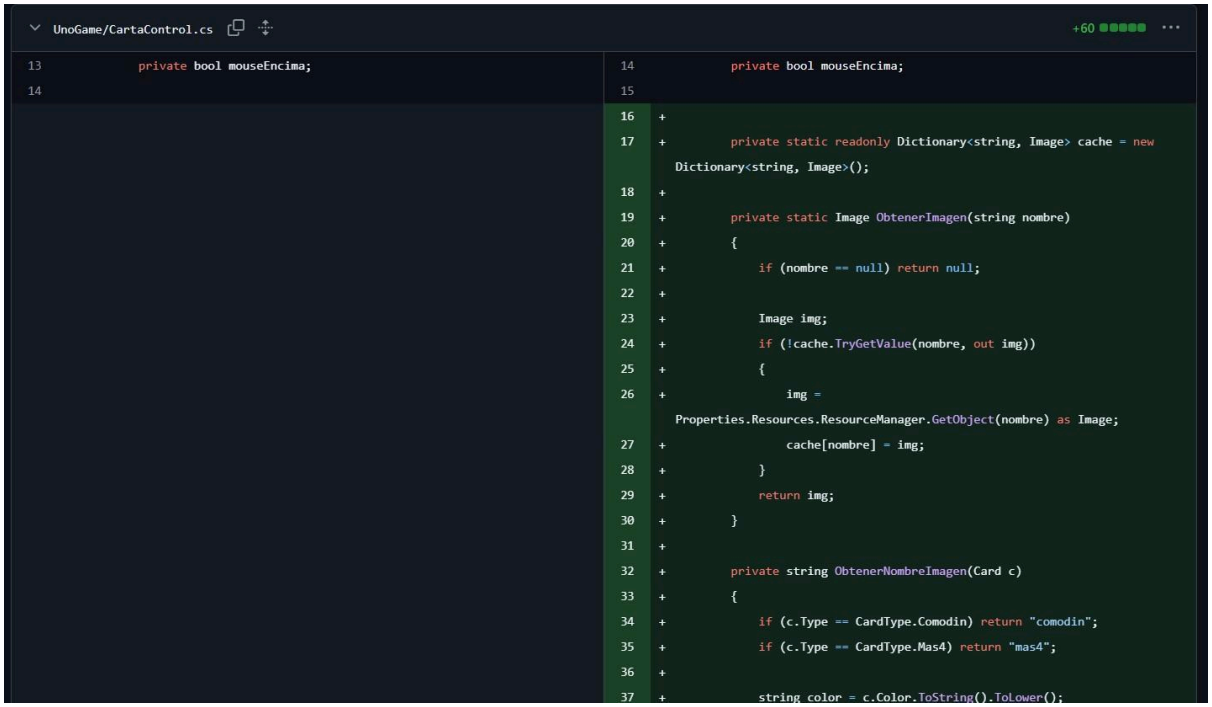




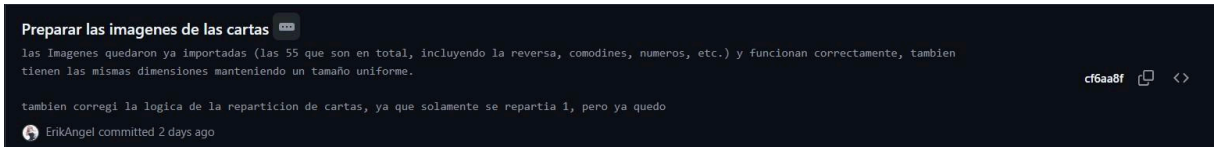
Oct 9, 2026

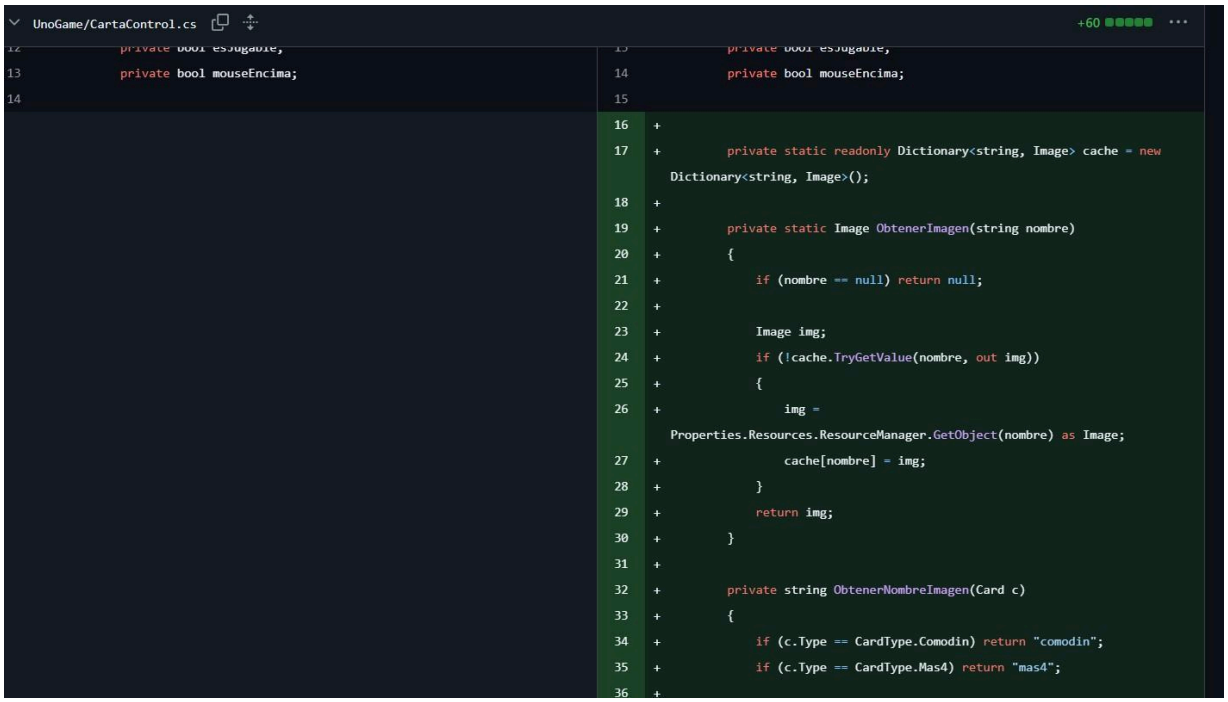
- **Merge pull request #52 from SergioZarateCas06/25-preparar-las-imagenes-de-las-cartas-v2 - ErikAngel**





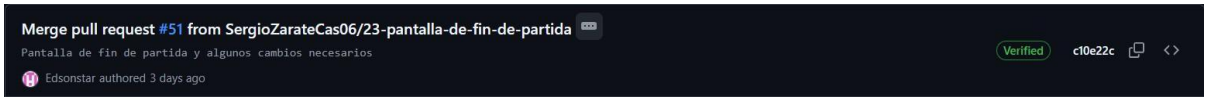
Preparar las imagenes de las cartas - ErikAngel





Oct 8, 2026

- **Merge pull request #51 from SergioZarateCas06/23-pantalla-de-fin-de-partida - Edsonstar**



```
51              TurnoCambiado += MostrarTurno;
52              RepartirCartas();
53          }
54          void iniciarTamanoDeBotones()
55          {
```

```
53              TurnoCambiado += MostrarTurno;
54              RepartirCartas();
55          }
56  +
57  +       void Form1_FormClosing(object sender, FormClosingEventArgs e)
58  +       {
59  +           if(cierreForzado)
60  +           {
61  +               return;
62  +           }
63  +           if (e.CloseReason == CloseReason.UserClosing)
64  +           {
65  +               DialogResult res = MessageBox.Show("Quieres salir del
    juego? Se perdera el progreso", "Salir del juego",
    MessageBoxButtons.YesNo, MessageBoxIcon.Warning);
66  +               if(res == DialogResult.No)
67  +               {
68  +                   e.Cancel = true;
69  +               }
70  +           }
71  +       }
72  +
73          void iniciarTamanoDeBotones()
74          {
```

- **Pantalla de fin de partida y algunos cambios necesarios - Edsonstar**

Pantalla de fin de partida y algunos cambios necesarios

Para la pantalla del final de la partida se necesito crear otro form donde se muestre quien gano, y dos botones, uno para volver a jugar, regresando al form1 e iniciando la partida desde cero, y el boton para volver al menu, mandando directamente al menu principal, los botones son imagenes a excepcion del texto que es un label, tambien se corrigieron cambios relacionados a las ventanas del programa, esto para que no se pueda modificar su tamaño al gusto, ademas de configurar los botones de minimizar, maximizar y cerrar, evitando el uso de maximizar y el de cerrar haciendo que tenga una operacion en form1 para no cerrar el juego de una, mientras que en los otros form se hace lo equivalente a su boton salir/regresar (a excepcion de configuracion).

16054a3

Edsonstar committed 3 days ago

```
53          }
```

```
55          }
56  +
57  +       void Form1_FormClosing(object sender, FormClosingEventArgs e)
58  +       {
59  +           if(cierreForzado)
60  +           {
61  +               return;
62  +           }
63  +           if (e.CloseReason == CloseReason.UserClosing)
64  +           {
65  +               DialogResult res = MessageBox.Show("Quieres salir del
    juego? Se perdera el progreso", "Salir del juego",
    MessageBoxButtons.YesNo, MessageBoxIcon.Warning);
66  +               if(res == DialogResult.No)
67  +               {
68  +                   e.Cancel = true;
69  +               }
70  +           }
71  +       }
72  +
```

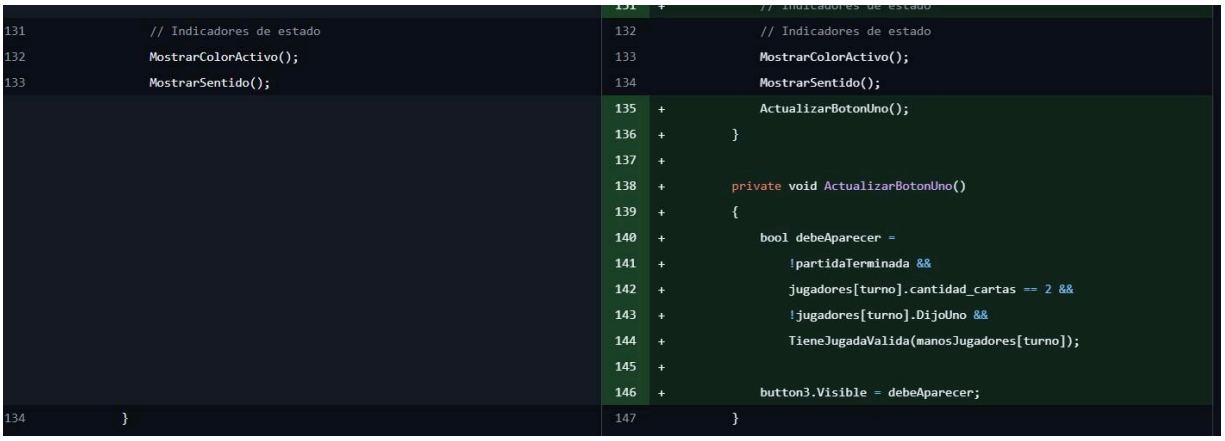
- **Merge pull request #50 from SergioZarateCas06/22-boton-uno-y-mensajes-de-estado - SergioZarateCas06**

Merge pull request #50 from SergioZarateCas06/22-boton-uno-y-mensajes-de-estado

Boton Uno y mensajes de accion

Verified 10777c3

SergioZarateCas06 authored 3 days ago

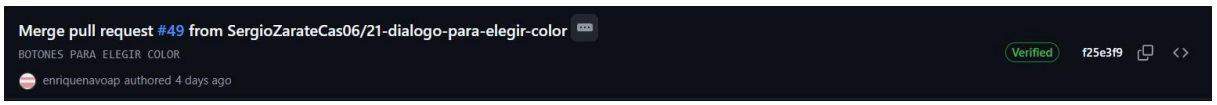


- **Boton Uno y mensajes de accion -SergioZarateCas06**

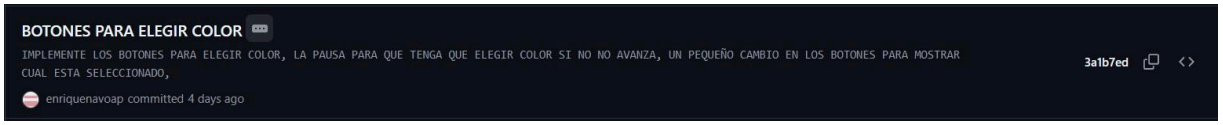


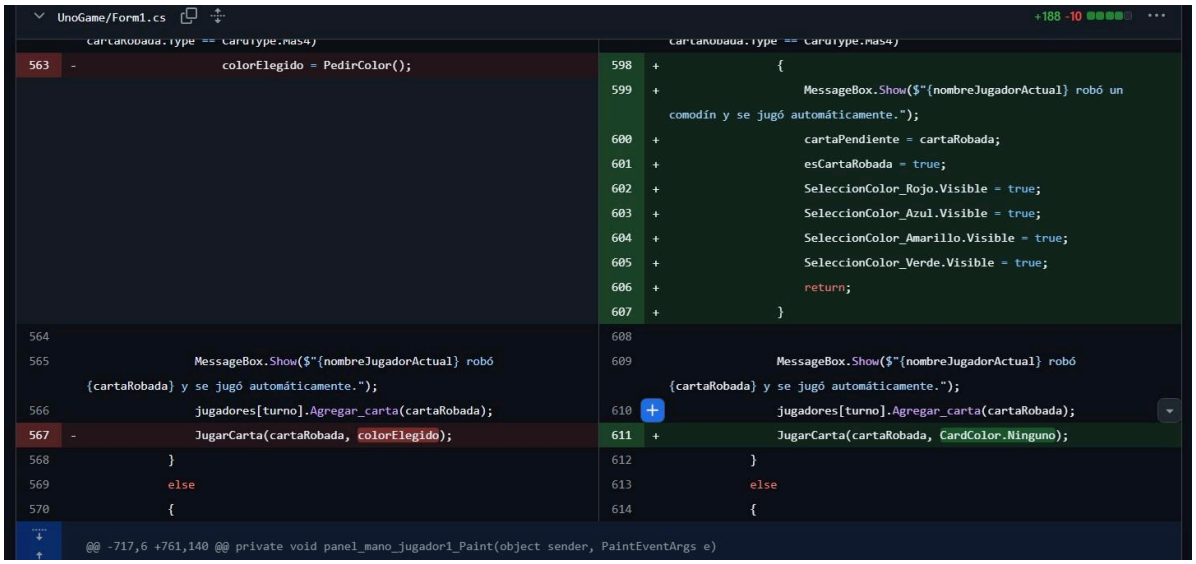
Oct 7, 2026

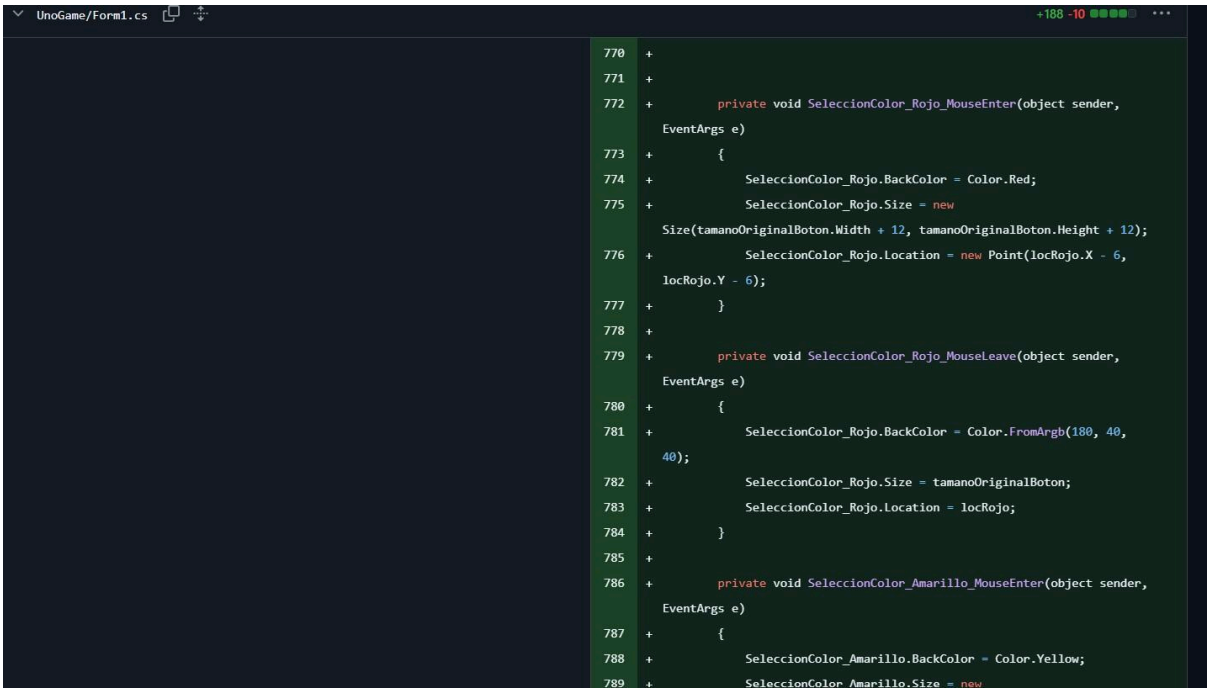
- **Merge pull request #49 from SergioZarateCas06/21-dialogo-para-elegir-color - enriquenavoap**



- **BOTONES PARA ELEGIR COLOR - enriquenavoap**



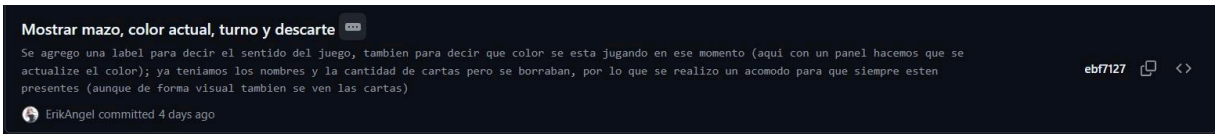


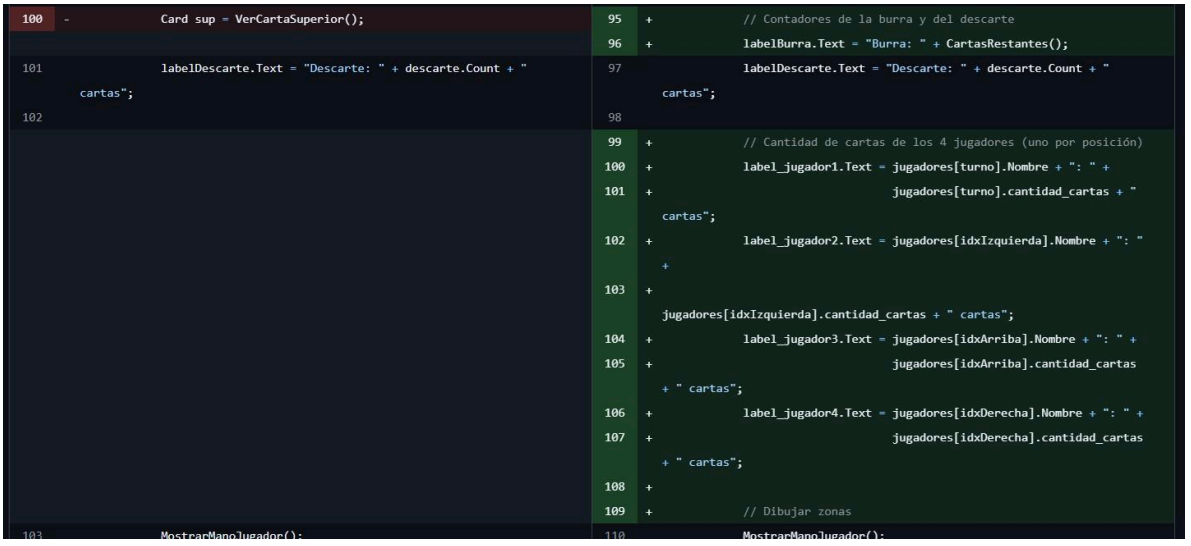


- **Merge pull request #48 from SergioZarateCas06/20-mostrar-mazo-descarte-color-actual-y-turno - ErikAngel**



- **Mostrar mazo, color actual, turno y descarte - ErikAngel**





```
109         }
110
111         private void MostrarRival(Panel panel, int indiceJugador)
```

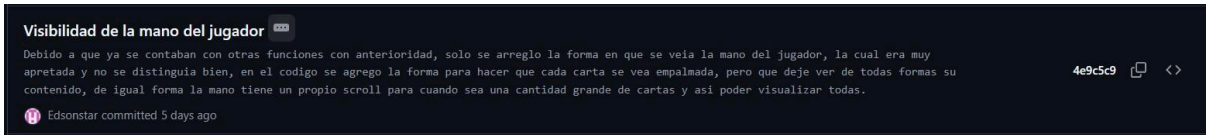
```
121 +
122 +       private void MostrarColorActivo()
123 +       {
124 +           Color c;
125 +           switch (colorActivo)
126 +           {
127 +               case CardColor.Rojo: c = Color.Red; break;
128 +               case CardColor.Azul: c = Color.RoyalBlue; break;
129 +               case CardColor.Verde: c = Color.Green; break;
130 +               case CardColor.Amarillo: c = Color.Gold; break;
131 +               default: c = Color.Gray; break;
132 +           }
133 +           panelColor.BackColor = c;
134 +           labelColor.Text = "Color actual: " + colorActivo;
135 +       }
136 +
137 +       private void MostrarSentido()
138 +       {
139 +           if (sentido == 1)
140 +               labelSentido.Text = "Sentido: Izquierda";
141 +           else
142 +               labelSentido.Text = "Sentido: Derecha";
143         }
144
145         private void MostrarRival(Panel panel, int indiceJugador)
```

Oct 6, 2026

- **Merge pull request #47 from SergioZarateCas06/19-mostrar-y-seleccionar-cartas-de-la-mano - Edsonstar**



- **Visibilidad de la mano del jugador - Edsonstar**



UnoGame/Form1.cs +32 -1

```
132             List<Card> mano = manosJugadores[turno];
133
```

```
132             List<Card> mano = manosJugadores[turno];
133
134 +           int anchoCarta = 70;
135 +           int separacion = 75;
136 +           int separacionMinima = 40;
137 +
138 +           int espacioDisponible = panel_mano_jugador1.Width -
    anchoCarta;
139 +
140 +           int separacionReal;
141 +
142 +           if(mano.Count > 1)
143 +           {
144 +               separacionReal = Math.Min(separacion, espacioDisponible
    / (mano.Count - 1));
145 +               if(separacionReal < separacionMinima)
146 +               {
147 +                   separacionReal = separacionMinima;
148 +               }
149 +           }
150 +           else
151 +           {
152 +               separacionReal = separacion;
153 +           }
154 +
```

- **Merge pull request #46 from SergioZarateCas06/18-control-de-usuario-para-dibujar-una-carta - SergioZarateCas06**

Merge pull request #46 from SergioZarateCas06/18-control-de-usuario-para-dibujar-una-carta

18 control de usuario para dibujar una carta

SergioZarateCas06 authored 5 days ago

Verified a456126

- **Cartas de cada jugador - SergioZarateCas06**

Cartas de cada jugador

Los paneles ahora son llenados con cartas, provienen de CardControl, un nuevo elemento del tipo Windows Forms control de usuario, aplica los metodos ya antes hechos para la jugabilidad

SergioZarateCas06 committed 5 days ago

1b4cce1

UnoGame/CartaControl.cs +198

```
@@ -0,0 +1,198 @@
```

```
1   + using System;
2   + using System.Drawing;
3   + using System.Drawing.Drawing2D;
4   + using System.Windows.Forms;
5   +
6   + namespace UnoGame
7   + {
8   +     public partial class CartaControl : UserControl
9   +     {
10  +         private Card carta;
11  +         private bool bocaAbajo;
12  +         private bool esJugable;
13  +         private bool mouseEncima;
14  +
15  +         public CartaControl()
16  +         {
17  +             InitializeComponent();
18  +             this.DoubleBuffered = true;
19  +             this.Size = new Size(70, 100);
20  +             this.Cursor = Cursors.Hand;
21  +
```

- **Cambio de las imagenes en resoruces - SergioZarateCas06**



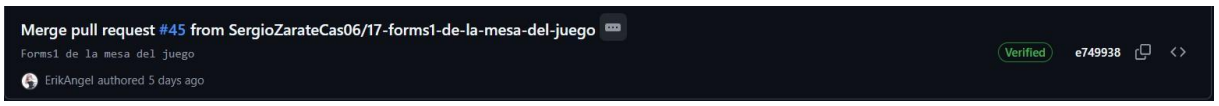
cambio de nombre a las recursues - SergioZarateCas06



- **PictureBox - SergioZarateCas06**

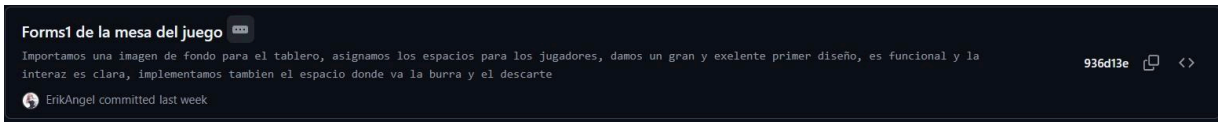


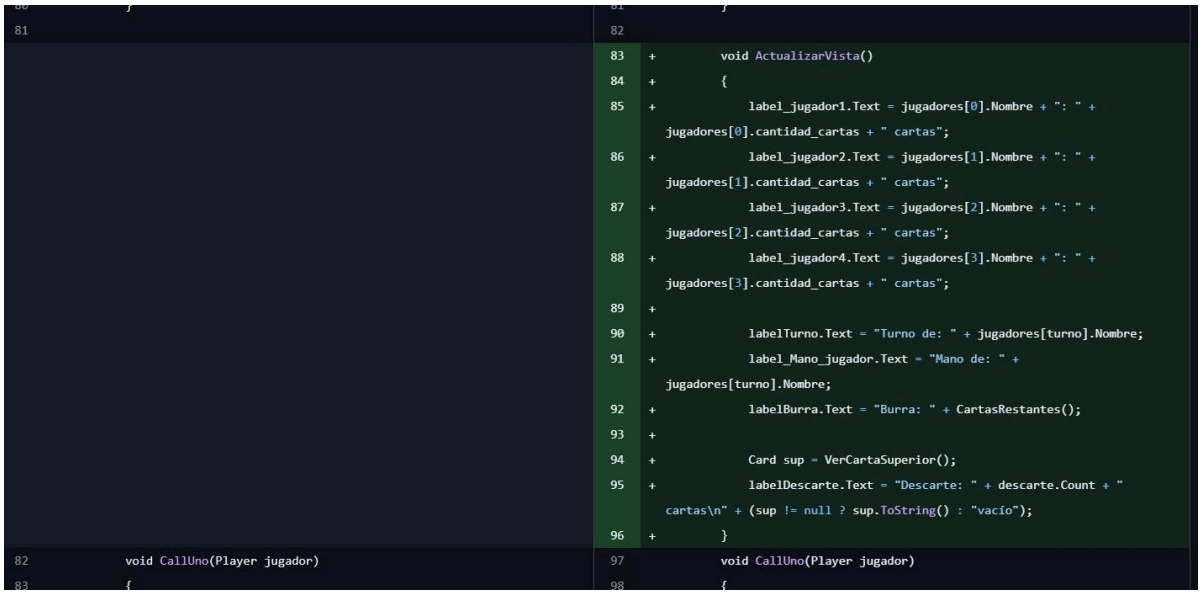
- **Merge pull request #45 from SergioZarateCas06/17-forms1-de-la-mesa-del-juego - ErikAngel**



Oct 5, 2026

- **Forms1 de la mesa del juego - ErikAngel**

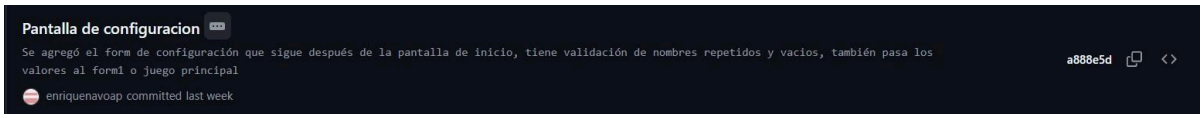


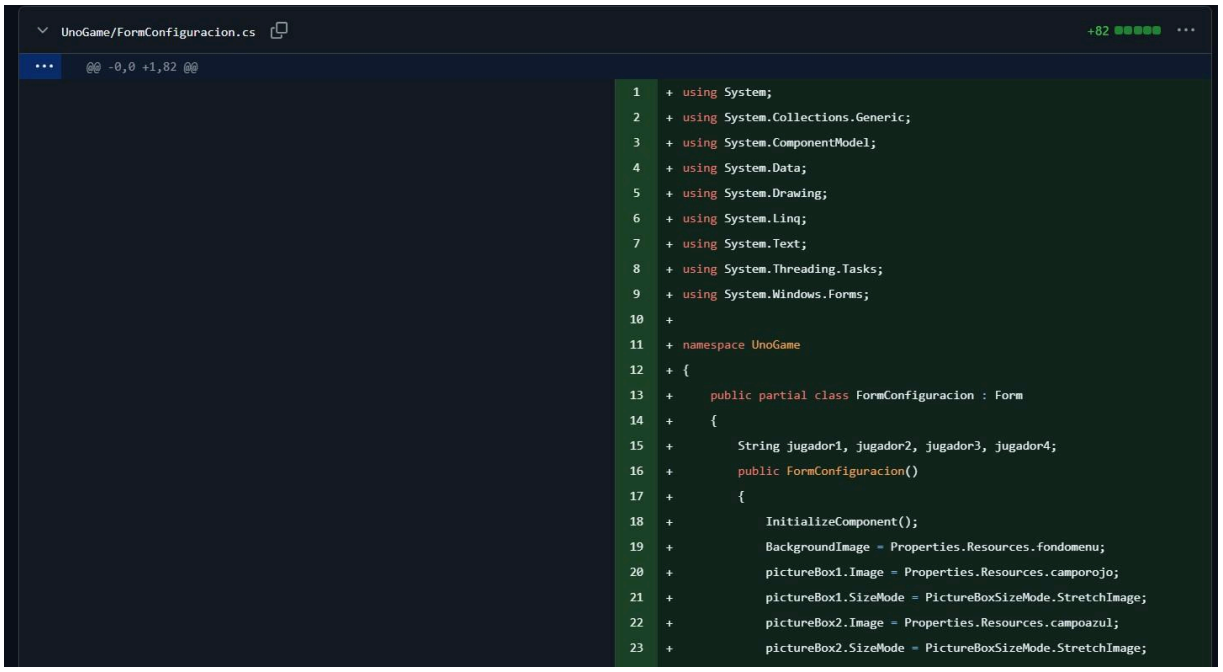


- **Merge pull request #44 from SergioZarateCas06/16-pantalla-de-configuracion-de-partida - enriquenavoap**



- **Pantalla de configuracion - enriquenavoap**





- **Merge pull request #43 from SergioZarateCas06/15-menu-principal - Edsonstar**



- **Interfaz del Menú Principal - Edsonstar**



UnoGame/FormHistorial.cs +20

```
@@ -0,0 +1,20 @@
+ using System;
+ using System.Collections.Generic;
+ using System.ComponentModel;
+ using System.Data;
+ using System.Drawing;
+ using System.Linq;
+ using System.Text;
+ using System.Threading.Tasks;
+ using System.Windows.Forms;
+
+ namespace UnoGame
+ {
+     public partial class FormHistorial : Form
+     {
+         public FormHistorial()
+         {
+             InitializeComponent();
+         }
+     }
+ }
```

- **Merge pull request #42 from SergioZarateCas06/15-menu-principal - Edsonstar**

Merge pull request #42 from SergioZarateCas06/15-menu-principal

Separar la clase Card del Form1

Edsonstar authored last week

Verified 2131253

- **Separar la clase Card del Form1 - Edsonstar**

Separar la clase Card del Form1

Para trabajar directamente con el diseñador del Form1 se debio de separar la clase Card del Form1 para que se pueda abrir, el funcionamiento del juego no cambio y funciona sin problemas

Edsonstar committed last week

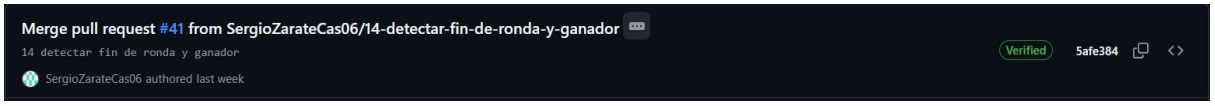
d56c498

UnoGame/Card.cs +46

```
+ using System;
+ using System.Collections.Generic;
+ using System.Linq;
+ using System.Text;
+ using System.Threading.Tasks;
+
+ namespace UnoGame
+ {
+     public enum CardColor { Rojo, Azul, Verde, Amarillo, Ninguno };
+     public enum CardType { Numero, Salta, Reversa, Mas2, Comodin, Mas4 };
+
+     public class Card
+     {
+         public int Number;
+         public CardColor Color;
+         public CardType Type;
+
+         public Card(int cardNumber, CardColor cardColor, CardType cardType)
+         {
+             this.Number = cardNumber;
+             this.Color = cardColor;
+             this.Type = cardType;
```

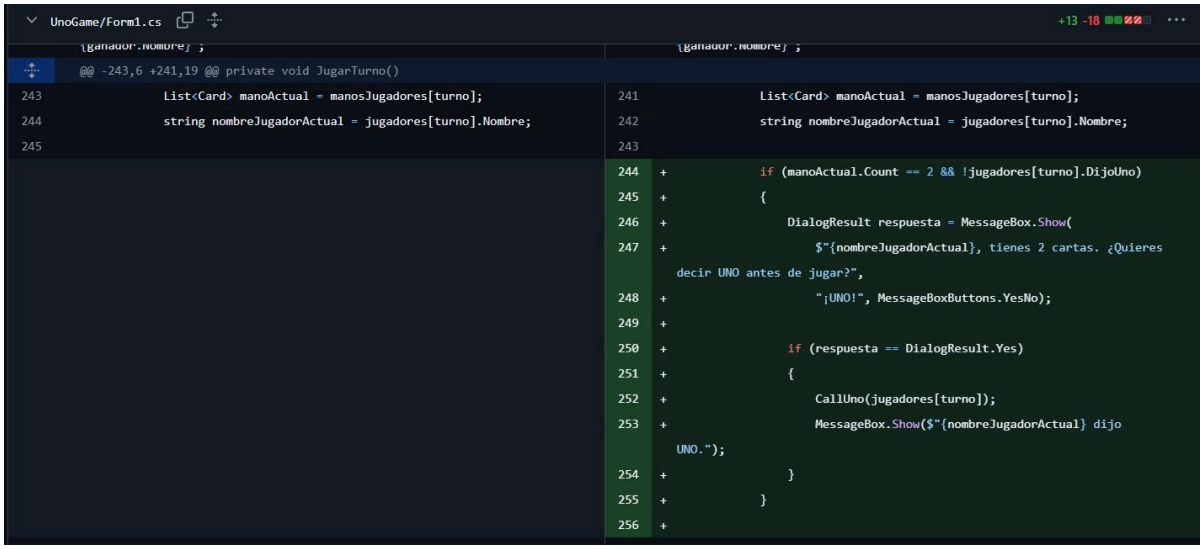
Oct 4, 2026

- **Merge pull request #41 from SergioZarateCas06/14-detectar-fin-de-ronda-y-ganador - SergioZarateCas06**



- **Boton Uno - SergioZarateCas06**





- **metodo actualizarbotonUno - SergioZarateCas06**



- **boton uno eliminado - SergioZarateCas06**



- **boton uno eliminado - SergioZarateCas06**



- **RepartirCartas - SergioZarateCas06**



```
@@ -404,6 +404,10 @@ public bool Quitar_carta(Card carta)
404         }                                                   404         }
405         private void RepartirCartas()                       405         private void RepartirCartas()
406         {                                                   406         {
                                                                407 +           partidaTerminada = false;
                                                                408 +           if (botonUno != null)
                                                                409 +               botonUno.Visible = false;
                                                                410 +
407             Cartas1.Clear();                                411             Cartas1.Clear();
408             Cartas2.Clear();                                412             Cartas2.Clear();
409             Cartas3.Clear();                                413             Cartas3.Clear();
```

- **bloqueo de jugarTurno - SergioZarateCas06**



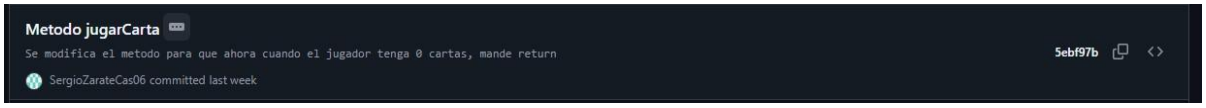

```
UnoGame/Form1.cs                                                                                    +6
@@ -249,6 +249,12 @@ private void TerminarPartida(Player ganador)
249                                                             249
250         private void JugarTurno()                           250         private void JugarTurno()
251         {                                                   251         {
                                                                252 +           if (partidaTerminada)
                                                                253 +           {
                                                                254 +               MessageBox.Show("La partida ya terminó. Inicia una
                                                                    nueva partida para seguir jugando.");
                                                                255 +               return;
                                                                256 +           }
                                                                257 +
252             List<Card> manoActual = manosJugadores[turno];  258             List<Card> manoActual = manosJugadores[turno];
253             string nombreJugadorActual = jugadores[turno].Nombre;  259             string nombreJugadorActual = jugadores[turno].Nombre;
254                                                             260
```

- **Metodo Nueva cerrarPartida - SergioZarateCas06**




```
UnoGame/Form1.cs                                                                                    +13
234             }                                               234             }
235         }                                                   235         }
236                                                             236
                                                                237 +
                                                                238 +       private void TerminarPartida(Player ganador)
                                                                239 +       {
                                                                240 +           partidaTerminada = true;
                                                                241 +           botonUno.Visible = false;
                                                                242 +
                                                                243 +           MessageBox.Show($"¡{ganador.Nombre} ganó la partida!", "Fin
                                                                    de la partida");
                                                                244 +           richTextBox1.Text = $"Partida terminada. Ganador:
                                                                    {ganador.Nombre}";
                                                                245 +
                                                                246 +           if (PartidaTerminada != null)
                                                                247 +               PartidaTerminada.Invoke(ganador);
                                                                248 +       }
                                                                249 +
237         private void JugarTurno()                           250         private void JugarTurno()
238         {                                                   251         {
239             List<Card> manoActual = manosJugadores[turno];  252             List<Card> manoActual = manosJugadores[turno];
```

- **Metodo jugarCarta - SergioZarateCas06**

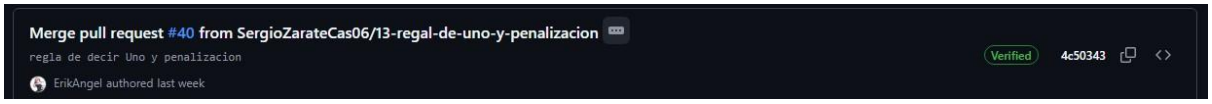


- **Nuevo evento - SergioZarateCas06**

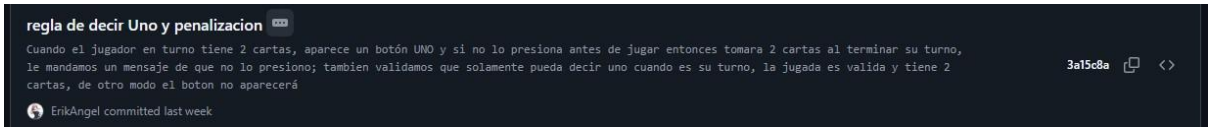


Oct 2, 2026

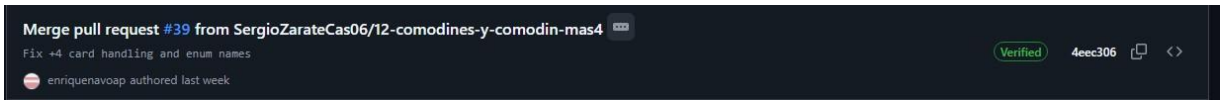
- **Merge pull request #40 from SergioZarateCas06/13-regal-de-uno-y-penalizacion - ErikAngel**



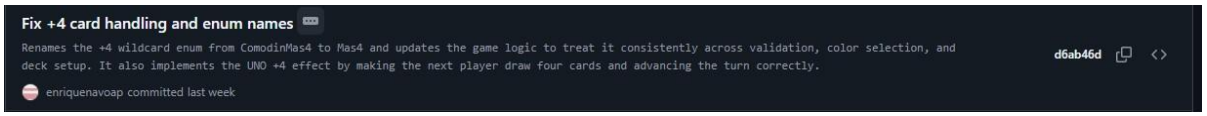
- **regla de decir Uno y penalizacion - ErikAngel**



- **Merge pull request #39 from SergioZarateCas06/12-comodines-y-comodin-mas4 - enriquenavoap**



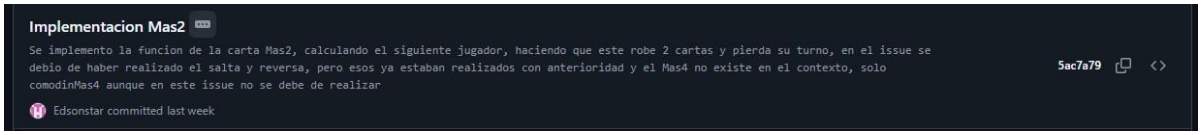
- **Fix +4 card handling and enum names - enriquenavoap**



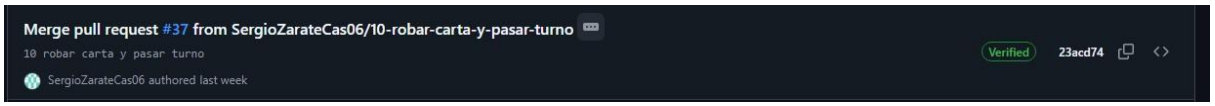
- **Merge pull request #38 from SergioZarateCas06/11-cartas-de-accion-salta-reversa-mas2-mas4 - Edsonstar**



- **Implementacion Mas2 - Edsonstar**



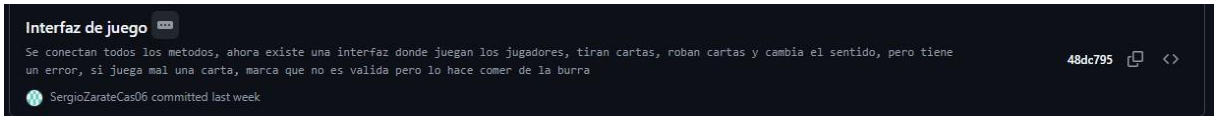
- **Merge pull request #37 from SergioZarateCas06/10-robar-carta-y-pasar-turno - SergioZarateCas06**



- **correcion de errores de penalizacion - SergioZarateCas06**



- **Interfaz de juego - SergioZarateCas06**

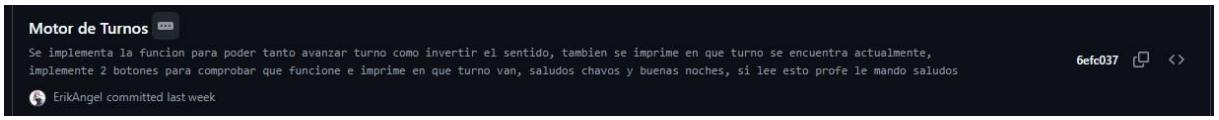


Oct 1, 2026

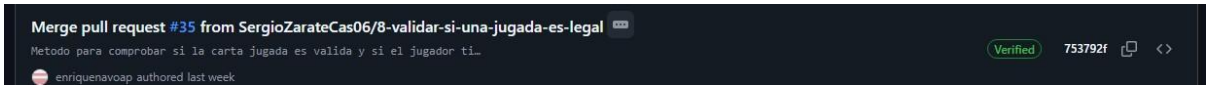
- **Merge pull request #36 from SergioZarateCas06/9-motor-de-turnos - ErikAngel**



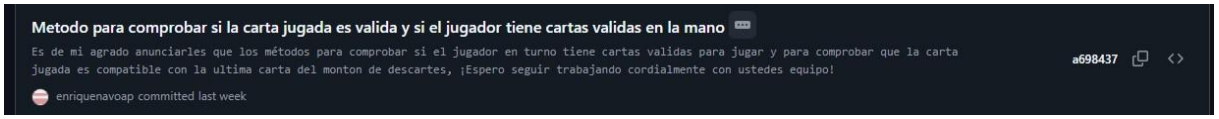
- **Motor de Turnos - ErikAngel**



- **Merge pull request #35 from SergioZarateCas06/8-validar-si-una-jugada-es-legal - enriquenavoap**



- **Metodo para comprobar si la carta jugada es válida y si el jugador tiene cartas válidas en la mano - enriquenavoap**



- **Cambio de tamaño de textbox - SergioZarateCas06**

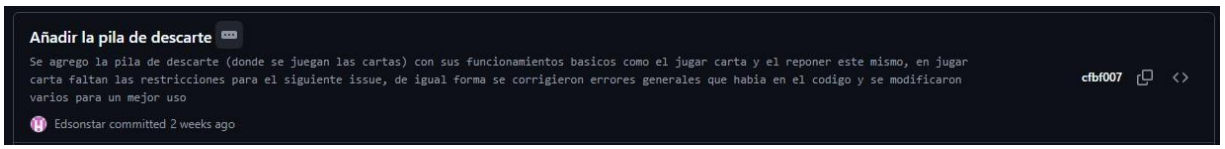


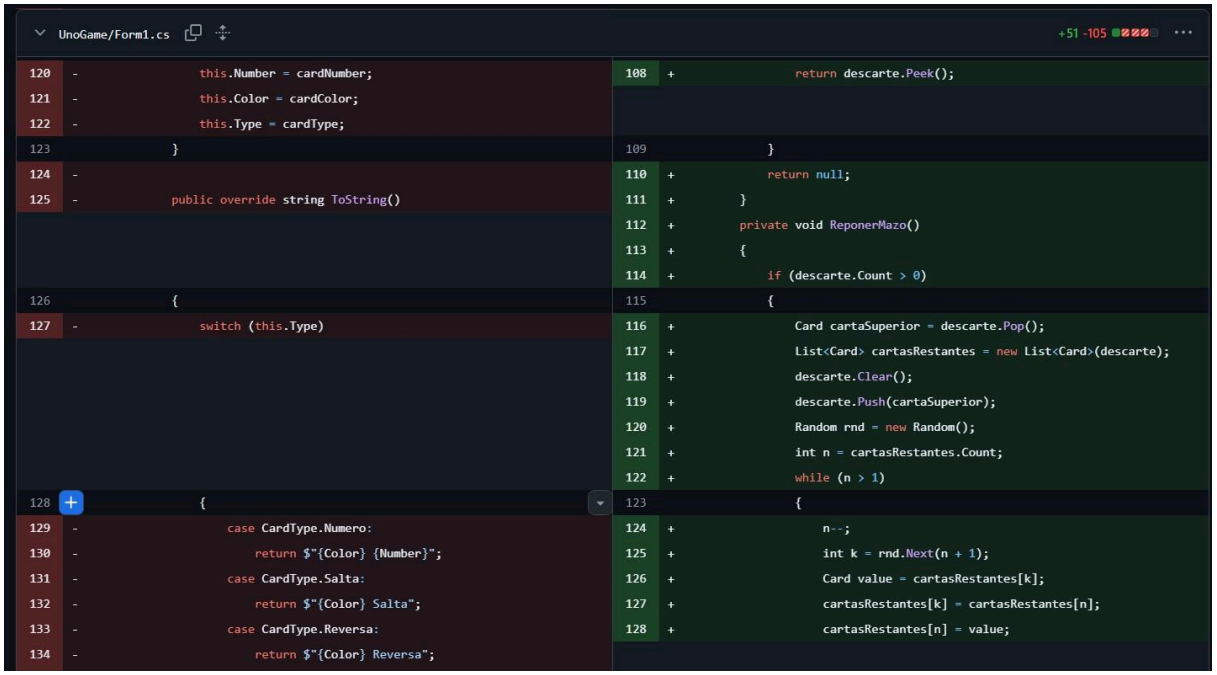
- **Merge pull request #34 from SergioZarateCas06/7-pila-de-cartas-descarte - Edsonstar**



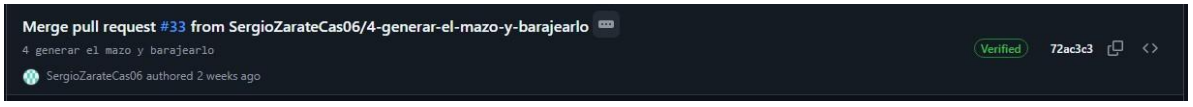
Sep 30, 2026

- **Añadir la pila de descarte - Edsonstar**





- **Merge pull request #33 from SergioZarateCas06/4-generar-el-mazo-y-barajearlo - SergioZarateCas06**



UnoGame/Form1.cs +103 -1

@@ -215,5 +215,107 @@ private void DevolverYBarajar(Card carta)

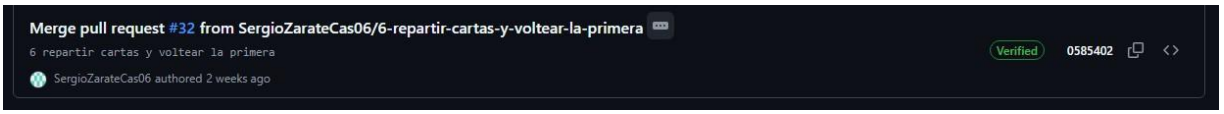
```
            foreach (Card c in restante)
                burra.Push(c);
        }
```

```
            foreach (Card c in restante)
                burra.Push(c);
        }
+
+       Card SacarCarta()
+       {
+           if (burra.Count > 0)
+           {
+               return burra.Pop();
+           }
+           return null;
+       }
+
+       int CartasRestantes()
+       {
+           return burra.Count;
+       }
+       private void BarajarMazo()
+       {
+           List<Card> mazo = new List<Card>();
+           CardColor[] colores = { CardColor.Rojo, CardColor.Azul,
CardColor.Verde, CardColor.Amarillo };
```

- **Merge pull request #31 from SergioZarateCas06/ErrorMerge31 - SergioZarateCas06**



- **Merge pull request #32 from SergioZarateCas06/6-repartir-cartas-y-voltear-la-primera - SergioZarateCas06**



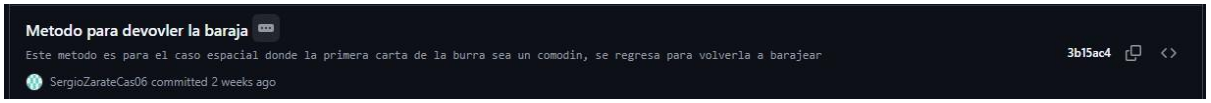
- **prueba - SergioZarateCas06**

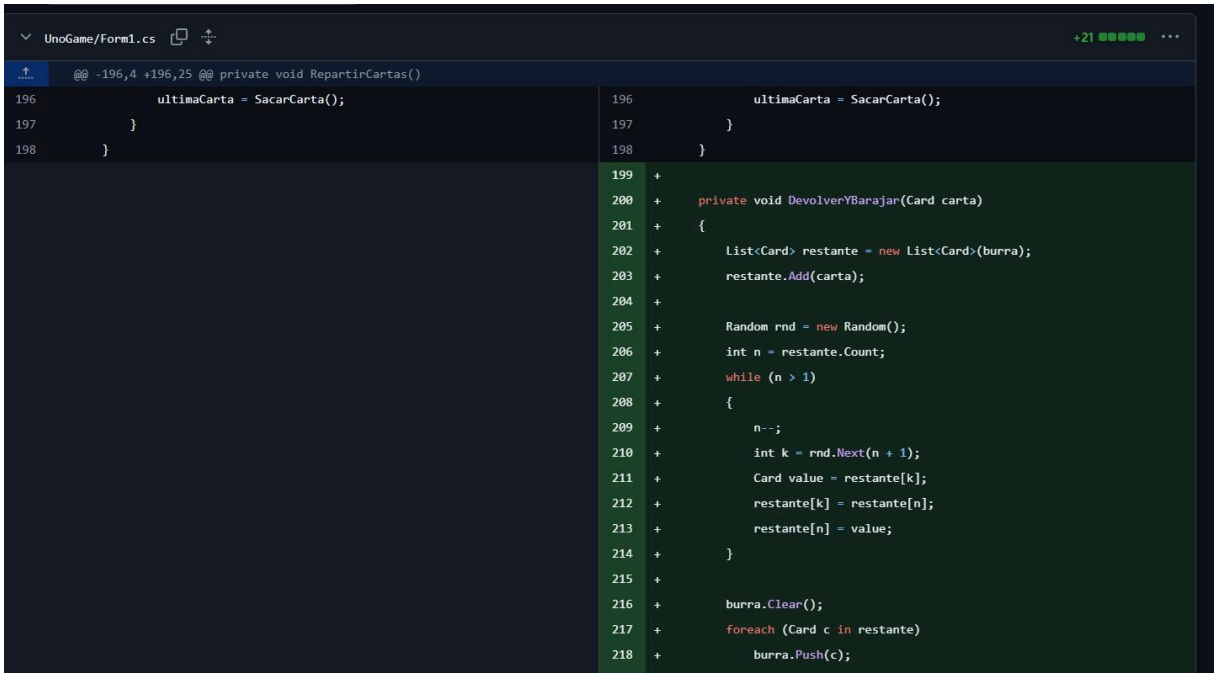



```
        private void button1_Click_1(object sender, EventArgs e)
        {
-
        }
-   }
```

```
        private void button1_Click_1(object sender, EventArgs e)
        {
+           RepartirCartas();
+           richTextBox1.Text =
+           $"J1: {string.Join(", ", Cartas1.Select(c =>
c.ToString()))}\n" +
+           $"J2: {string.Join(", ", Cartas2.Select(c =>
c.ToString()))}\n" +
+           $"J3: {string.Join(", ", Cartas3.Select(c =>
c.ToString()))}\n" +
+           $"J4: {string.Join(", ", Cartas4.Select(c =>
c.ToString()))}\n" +
+           $"Carta volteada: {ultimaCarta}\n" +
+           $"Cartas restantes en la burra: {CartasRestantes()}";
        }
```

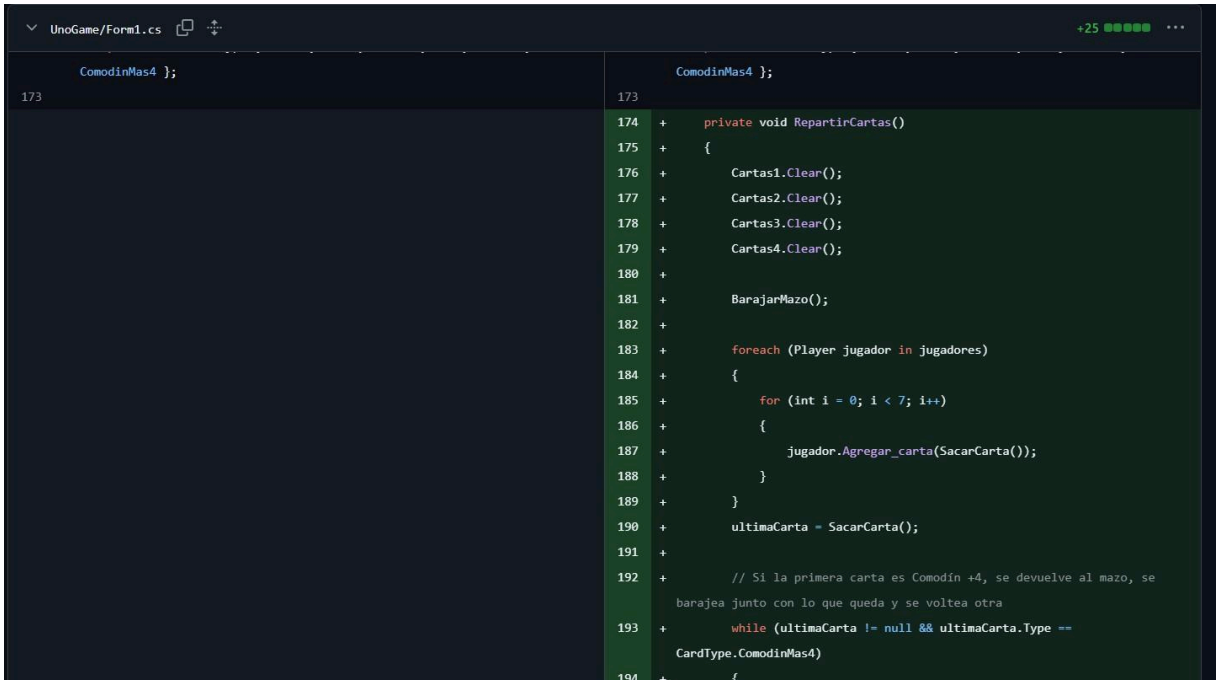
- **Metodo para devovler la baraja - SergioZarateCas06**





- **Metodo repartirCartas - SergioZarateCas06**



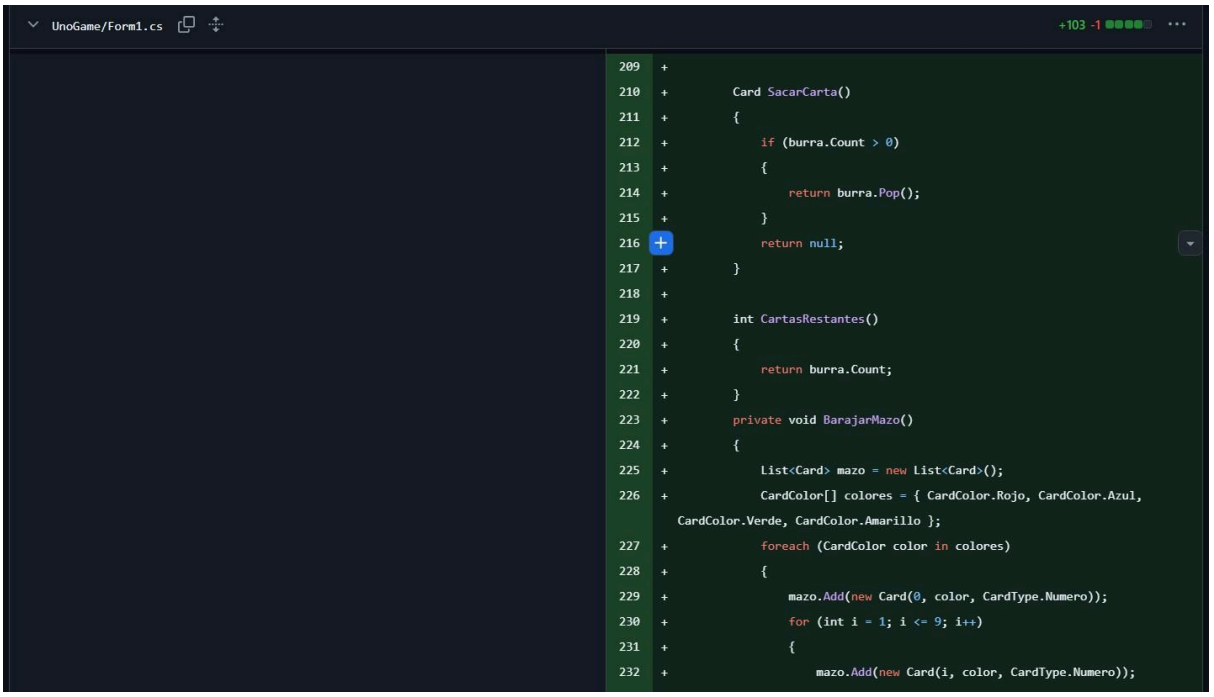


- **Error de duplicacion - SergioZarateCas06**



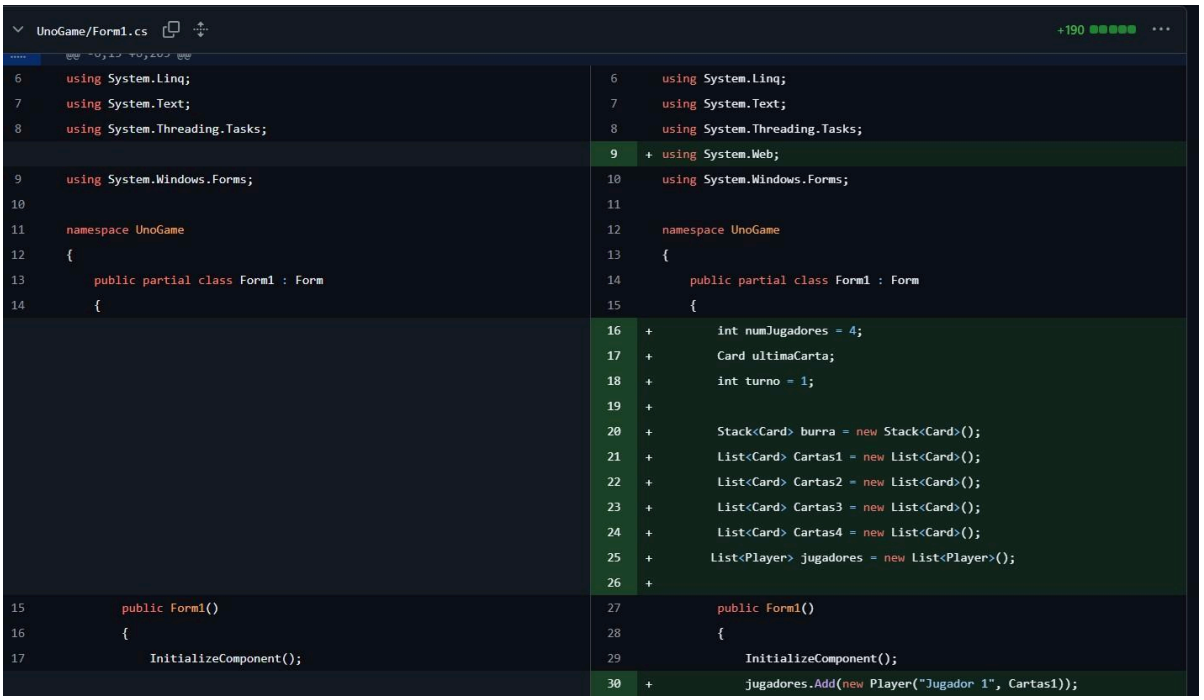
- **Merge branch ‘4-generar-el-mazo-y-barajearlo’ into ErrorMerge31 - enriquenavoap**





- **Merge branch ‘main’ into 6-repartir-cartas-y-voltear-la-primera - SergioZarateCas06**

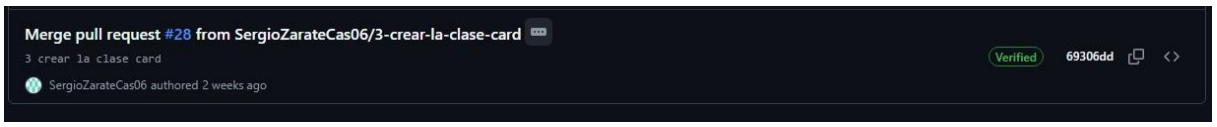




- **Merge pull request #30 from SergioZarateCas06/5-modelo-del-jugador - ErikAngel**



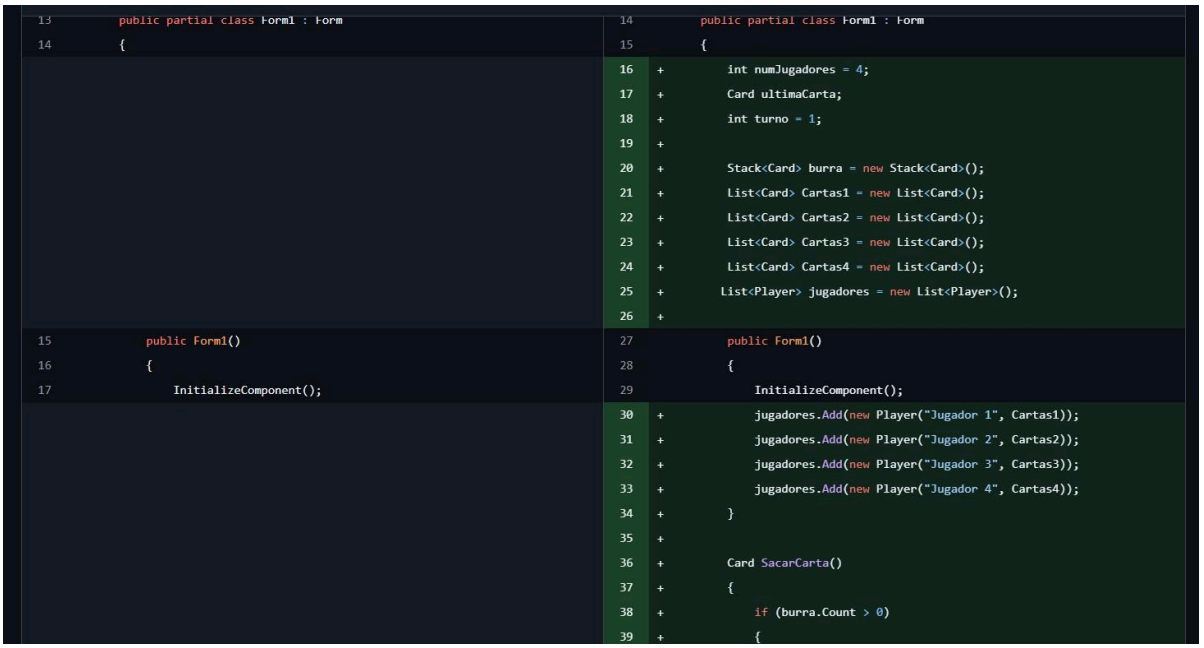
- **Merge pull request #28 from SergioZarateCas06/3-crear-la-clase-card - SergioZarateCas06**



Sep 29, 2026

- **ya quedo - ErikAngel**





```
13      public partial class Form1 : Form
14      {
15          public Form1()
16          {
17              InitializeComponent();
```

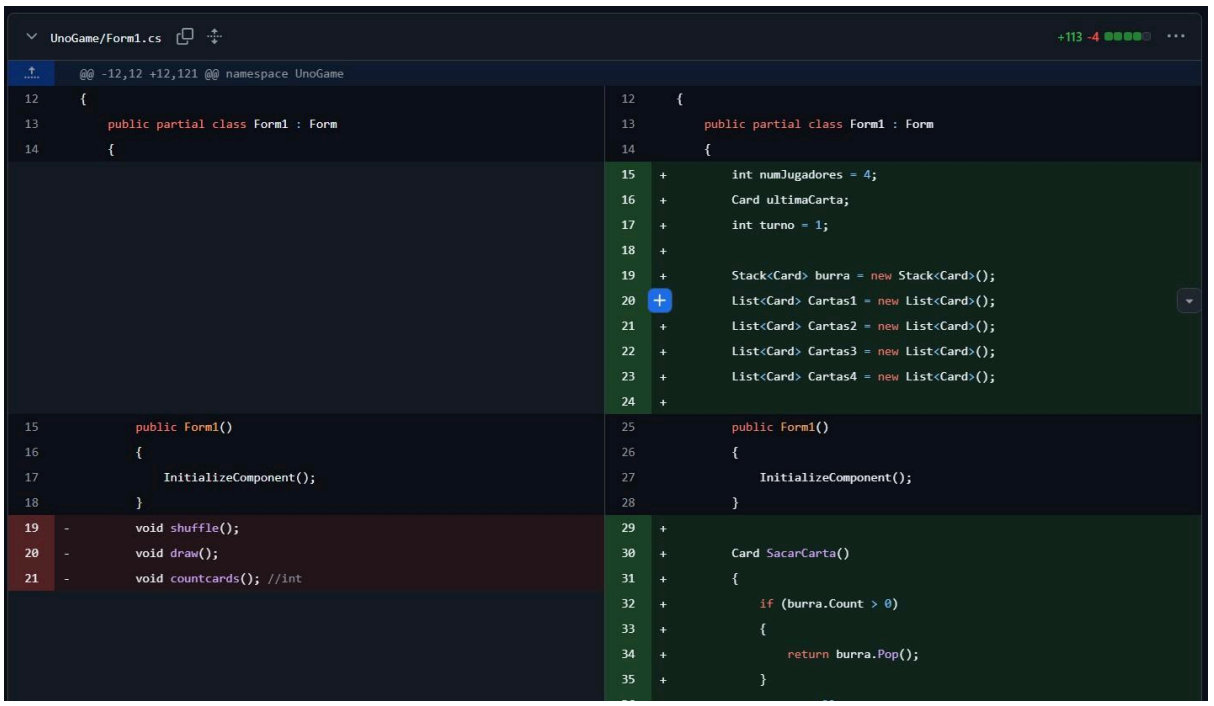
```
14      public partial class Form1 : Form
15      {
16  +       int numJugadores = 4;
17  +       Card ultimaCarta;
18  +       int turno = 1;
19  +
20  +       Stack<Card> burra = new Stack<Card>();
21  +       List<Card> Cartas1 = new List<Card>();
22  +       List<Card> Cartas2 = new List<Card>();
23  +       List<Card> Cartas3 = new List<Card>();
24  +       List<Card> Cartas4 = new List<Card>();
25  +      List<Player> jugadores = new List<Player>();
26  +
27          public Form1()
28          {
29              InitializeComponent();
30  +           jugadores.Add(new Player("Jugador 1", Cartas1));
31  +           jugadores.Add(new Player("Jugador 2", Cartas2));
32  +           jugadores.Add(new Player("Jugador 3", Cartas3));
33  +           jugadores.Add(new Player("Jugador 4", Cartas4));
34  +       }
35  +
36  +       Card SacarCarta()
37  +       {
38  +           if (burra.Count > 0)
39  +           {
```

Sep 26, 2026

- **4 - enriquenavoap**



```
4
ya
enriquenavoap committed 2 weeks ago
82cf3c4
```



```
UnoGame/Form1.cs                                                    +113 -4
@@ -12,12 +12,121 @@ namespace UnoGame
12      {
13          public partial class Form1 : Form
14          {
15          public Form1()
16          {
17              InitializeComponent();
18          }
19  -       void shuffle();
20  -       void draw();
21  -       void countcards(); //int
```

```
12      {
13          public partial class Form1 : Form
14          {
15  +       int numJugadores = 4;
16  +       Card ultimaCarta;
17  +       int turno = 1;
18  +
19  +       Stack<Card> burra = new Stack<Card>();
20  +       List<Card> Cartas1 = new List<Card>();
21  +       List<Card> Cartas2 = new List<Card>();
22  +       List<Card> Cartas3 = new List<Card>();
23  +       List<Card> Cartas4 = new List<Card>();
24  +
25          public Form1()
26          {
27              InitializeComponent();
28          }
29  +
30  +       Card SacarCarta()
31  +       {
32  +           if (burra.Count > 0)
33  +           {
34  +               return burra.Pop();
35  +           }
```

Sep 23, 2026

- **Update Form1.cs - Edsonstar**



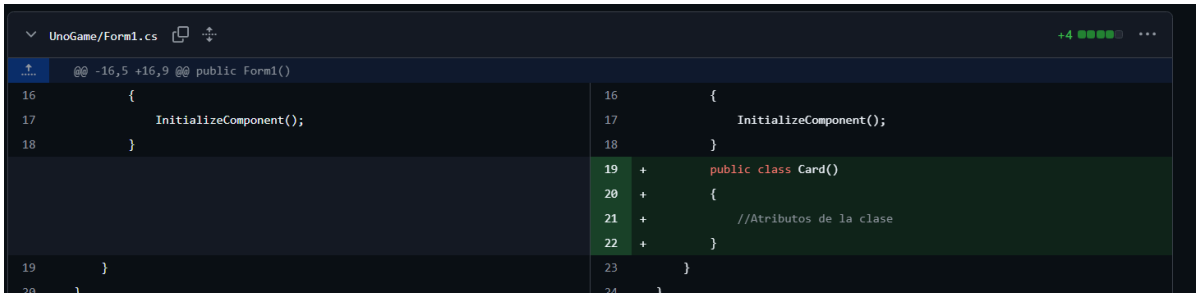
Sep 22, 2026

- **Métodos - enriquenavoap**



- **Definición de la clase Card - Edsonstar**





- **Update README.md - ErikAngel**

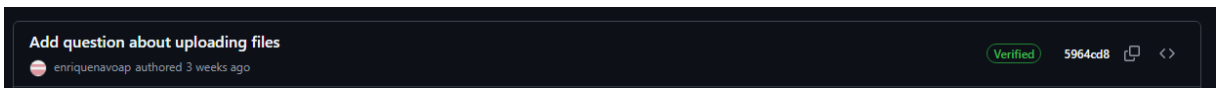


- **Creación metodo repartirCartas - SergioZarateCas06**

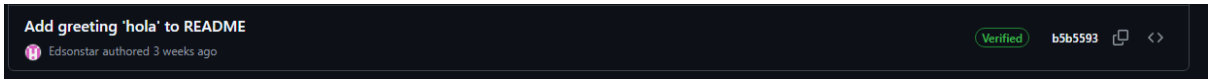


Sep 21, 2026

- **Add question about uploading files - enriquenavoap**



- **Add greeting ‘hola’ to README - Edsonstar**



Sep 20, 2026

- **Update README.md - ErikAngel**



Sep 19, 2026

- **Add greeting message to README - enriquenavoap**



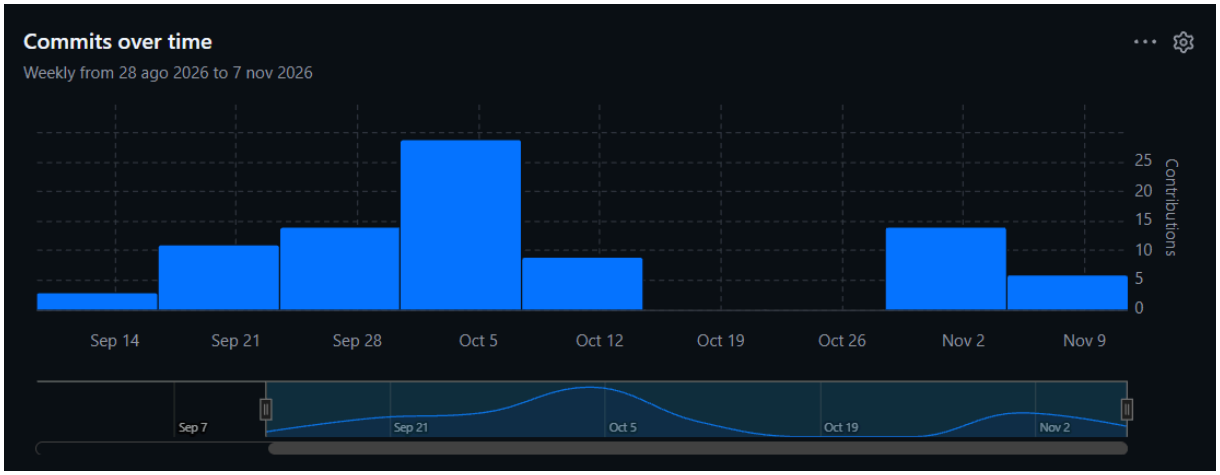
- **Creacion del proyecto UnoGame - SergioZarateCas06**

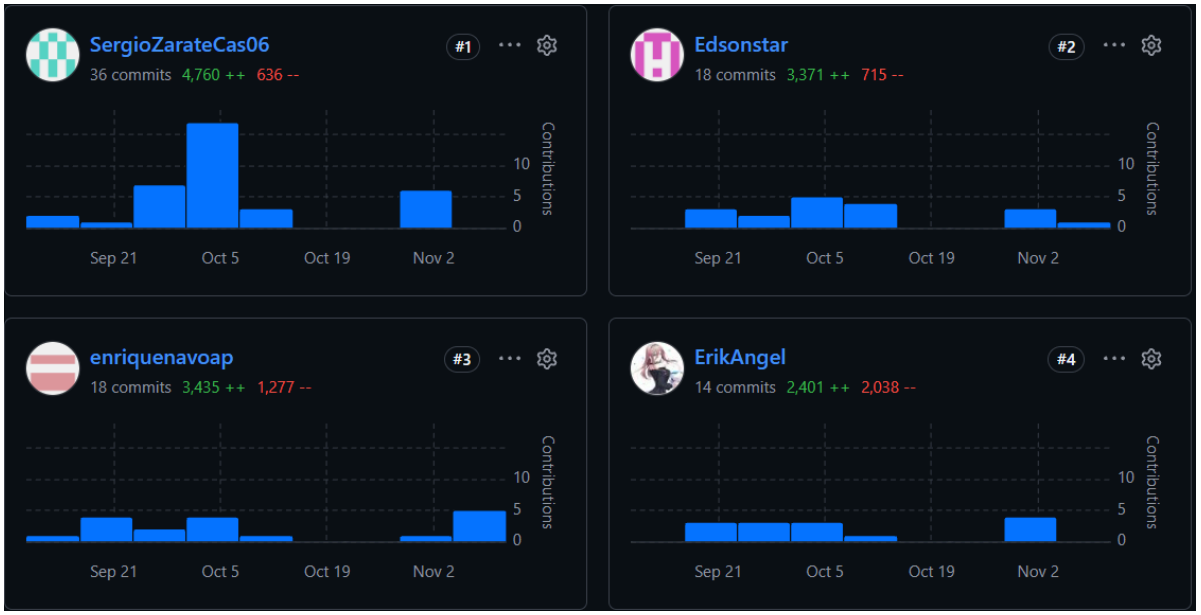


- **Estructura Inicial del Proyecto - SergioZarateCas06**



RESUMEN DE COMMITS





LISTADO DE RAMAS



- Lógica Base
    - Detectar fin de ronda y ganador - SergioZarateCas06
    - Regal de UNO y penalizacion - ErikAngel
    - Comodines y comodin Mas4 - enriquenavoap
    - Cartas de accion (salta, reversa, Mas2, Mas4) - Edsonstar
    - Robar carta y pasar turno - SergioZarateCas06
    - Motor de turnos - ErikAngel
    - Validar si una jugada es legal - enriquenavoap
    - Pila de Cartas (Descarte) - Edsonstar
    - Repartir cartas y voltear la primera - SergioZarateCas06
    - Modelo del Jugador - ErikAngel
    - Generar el mazo y barajearlo - enriquenavoap
    - Crear la clase Card - Edsonstar
- Interfaz
    - Sonidos y animaciones - enriquenavoap
    - Pantalla de fin de partida - Edsonstar
    - Boton UNO y mensajes de estado - SergioZarateCas06
    - Dialogo para elegir color - enriquenavoap
    - Mostrar mazo, descarte, color actual y turno - ErikAngel
    - Mostrar y seleccionar cartas de la mano - Edsonstar
    - Control de usuario para dibujar una carta - SergioZarateCas06
    - Forms1 de la mesa del juego - ErikAngel
    - Pantalla de configuracion de partida - enriquenavoap
    - Menu principal – Edsonstar

- Recursos e Integración
    - Api y Base de datos - SergioZarateCas06
    - Musica y silenciar - enriquenavoap
    - Pruebas y revision final - Edsonstar
    - Eventos del juego para el registro en base de datos - SergioZarateCas06
    - Preparar las imagenes de las cartas-v2 - ErikAngel
- Estructura de código
    - Limpieza de código y ordenamiento - enriquenavoap
    - Documentación de código - ErikAngel

- Base y contrato
    - Acordar y documentar el protocolo de mensajes - Edsonstar
    - Integrar UnoServer - SergioZarateCas06
    - Documentar la arquitectura cliente servidor en readme - SergioZarateCas06
- Servidor
    - Probar el motor de reglas - SergioZarateCas06
    - Sala de espera en el servidor - SergioZarateCas06
    - Procesar acciones y difundir el estado personalizado - SergioZarateCas06
    - Manejo de desconexiones y reinicio de sala - SergioZarateCas06
- Cliente
    - Pantalla de conexión - Edsonstar
    - Sala de espera en el cliente- Edsonstar
    - Form1 como vista - ErikAngel
    - Form1 envia las acciones al servidor - ErikAngel
    - Mostrar los eventos del servidor en pantalla - ErikAngel
    - Fin de partida y errores - Edsontar
    - Limpiar la logica local de form1 - ErikAngel
- Historial e Infraestructura
    - Historial por red - enriquenavoap
    - Guia para levantar el servidor - enriquenavoap
- Red y Pruebas
    - Configurar red y firewall - enriquenavoap
    - Prueba de partida completa con 4 clientes - enriquenavoap
    - Prueba de fallos - enriquenavoap
    - Actualizar el reporte - **EQUIPO**

Branch	Updated	Check status	Behind	Ahead	Pull request
86-prueba-de-fallos	5 hours ago		1	0	#107
85-prueba-de-partida-completa-con-4-clientes	6 hours ago		3	0	#106
84-configurar-red-y-firewall	6 hours ago		5	0	#105
83-guia-para-levantar-el-servidor	7 hours ago		7	0	#104
82-historial-por-red	7 hours ago		9	0	#103
80-fin-de-partida-y-errores	8 hours ago		11	0	#102
81-limpirar-la-logica-local-de-form1	19 hours ago		13	0	#101
79-mostrar-los-eventos-del-servidor-en-pantalla	yesterday		15	0	#100
78-form1-envia-las-acciones-al-servidor	yesterday		17	0	#99
77-form1-como-vista	yesterday		19	0	#98
76-sala-de-espera-en-el-cliente	yesterday		21	0	#97
75-pantalla-de-conexion	yesterday		23	0	#96
73-manejo-de-desconexiones-y-reinicio-de-sala	yesterday		26	0	#95
72-procesar-acciones-y-difundir-el-estado-personalizado	yesterday		29	0	#93
71-sala-de-espera-en-el-servidor	2 days ago		31	0	#92
70-probar-el-motor-de-reglas	2 days ago		33	0	#91
69-documentar-la-arquitectura-cliente-servidor-en-readme	2 days ago		35	0	#90
63-reporte	2 days ago		41	0	#89
68-integrar-unoserver	2 days ago		38	0	
67-acordar-y-documentar-el-protocolo-de-mensajes	3 days ago		39	0	#88

Branch	Updated	Check status	Behind	Ahead	Pull request
main	13 minutes ago		Default		
62-documentación-de-código	12 minutes ago		0	0	
61-limpieza-de-código-y-ordenamiento	15 minutes ago		2	0	
27-pruebas-y-revision-final	2 hours ago		4	0	#60
57-api-y-base-de-datos	19 hours ago		9	0	#58
26-eventos-del-juego-para-el-registro-en-base-de-datos	20 hours ago		11	0	#56
24-sonidos-y-animaciones	yesterday		16	0	#55
54-musica-y-silenciar	yesterday		16	0	
25-preparar-las-imagenes-de-las-cartas-v2	2 days ago		19	0	#52
23-pantalla-de-fin-de-partida	3 days ago		21	0	#51
22-boton-uno-y-mensajes-de-estado	3 days ago		23	0	#50
21-dialogo-para-elegir-color	4 days ago		25	0	#49
20-mostrar-mazo-descarte-color-actual-y-turno	4 days ago		27	0	#48
19-mostrar-y-seleccionar-cartas-de-la-mano	5 days ago		29	0	#47
18-control-de-usuario-para-dibujar-una-carta	5 days ago		31	0	#46
17-forms1-de-la-mesa-del-juego	last week		36	0	#45
16-pantalla-de-configuracion-de-partida	last week		38	0	#44
15-menu-principal	last week		41	0	#43
14-detectar-fin-de-ronda-y-ganador	last week		44	0	#41
13-regal-de-uno-y-penalizacion	last week		54	0	#40
9-motor-de-turnos	last week		55	2	
12-comodines-y-comodin-mas4	last week		56	0	#39
11-cartas-de-accion-salta-reversa-mas2-mas4	last week		58	0	#38
10-robar-carta-y-pasar-turno	last week		60	0	#37
8-validar-si-una-jugada-es-legal	last week		65	0	#35
2-ejemplo-erik-chat	last week		67	0	
7-pila-de-cartas-descarte	2 weeks ago		68	0	#34
4-generar-el-mazo-y-barajearlo	2 weeks ago		78	0	#33
6-repartir-cartas-y-voltear-la-primera	2 weeks ago		75	0	#32
ErrorMerge31	2 weeks ago		79	0	#31
3-crear-la-clase-card	2 weeks ago		84	1	
5-modelo-del-jugador	2 weeks ago		86	0	#30

CAPTURAS DEL FUNCIONAMIENTO DEL JUEGO

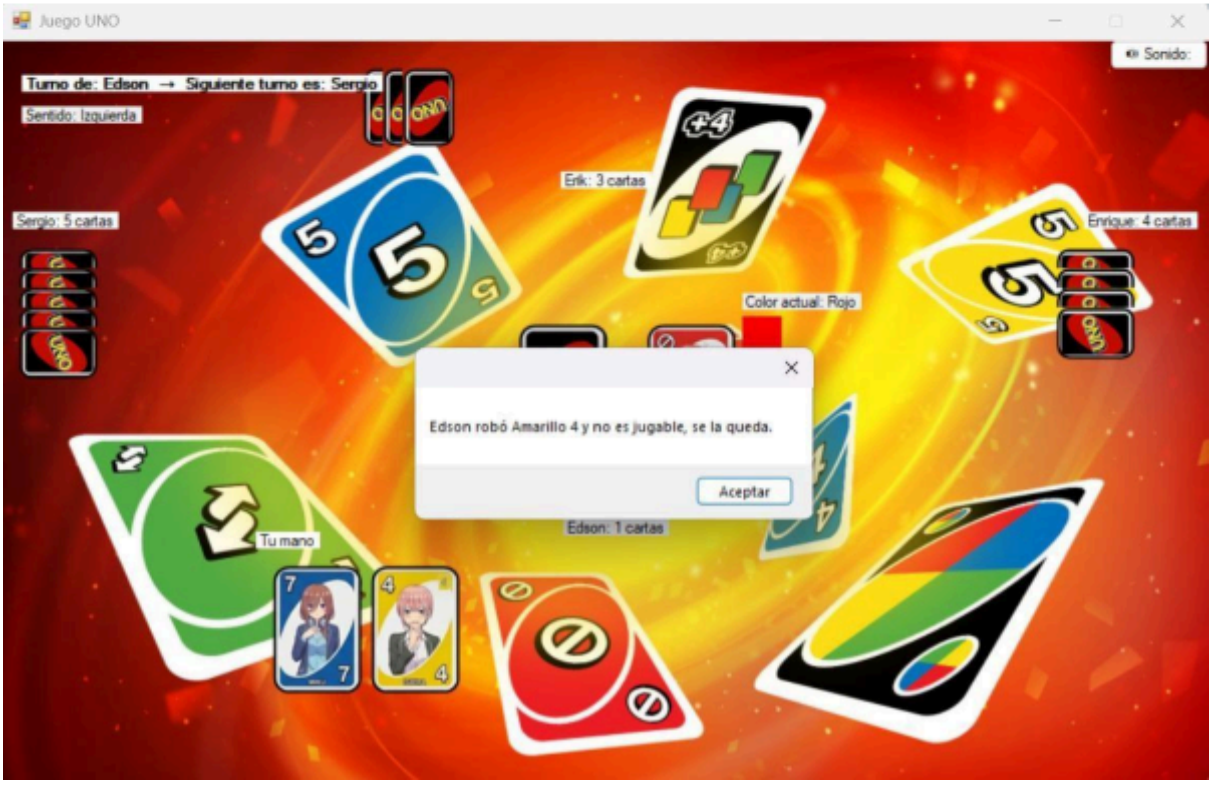





Juego UNO
Sonido:
Turno de: Edson → Siguiente turno es: Sergio
Sentido: Izquierda
Erik: 7 cartas
Sergio: 7 cartas
Enrique: 7 cartas
Color actual: Amarillo
Edson: 7 cartas
Tu mano


Juego UNO
Sonido:
Turno de: Edson → Siguiente turno es: Sergio
Sentido: Izquierda
Erik: 2 cartas
Sergio: 6 cartas
Enrique: 4 cartas
Color actual: Rojo
¡UNO!
Edson: 2 cartas
Tu mano


Juego UNO
Sonido:
Turno de: Erik → Siguiente turno es: Enrique
Sentido: Izquierda
Edson: 1 cartas
Enrique: 4 cartas
Sergio: 5 cartas
Color actual: Rojo
Erik no dijo UNO, come 2 cartas.
Aceptar
¡UNO!
Erik: 2 cartas
Tu mano

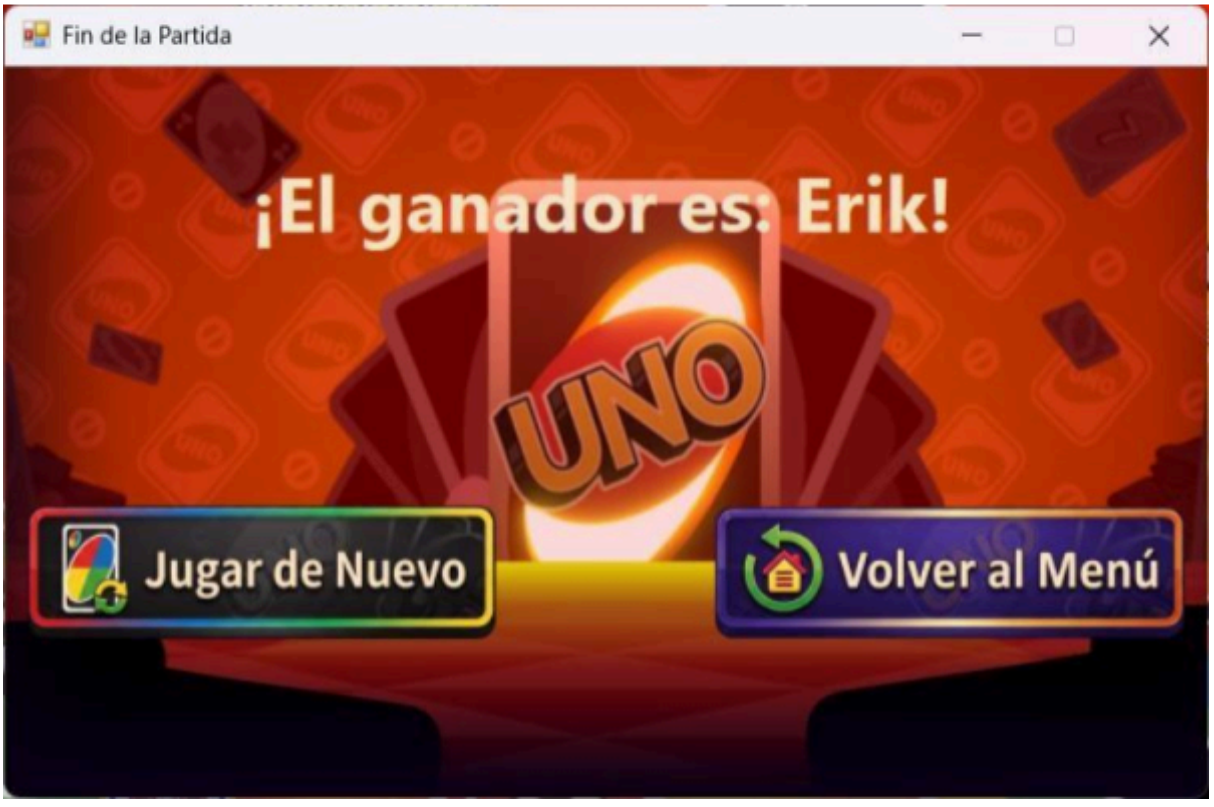

Juego UNO
Sonido:
Turno de: Edson → Siguiente turno es: Sergio
Sentido: Izquierda
Erik: 3 cartas
Sergio: 5 cartas
Enrique: 4 cartas
Color actual: Rojo
Edson robó Amarillo 4 y no es jugable, se la queda.
Aceptar
Edson: 1 cartas
Tu mano

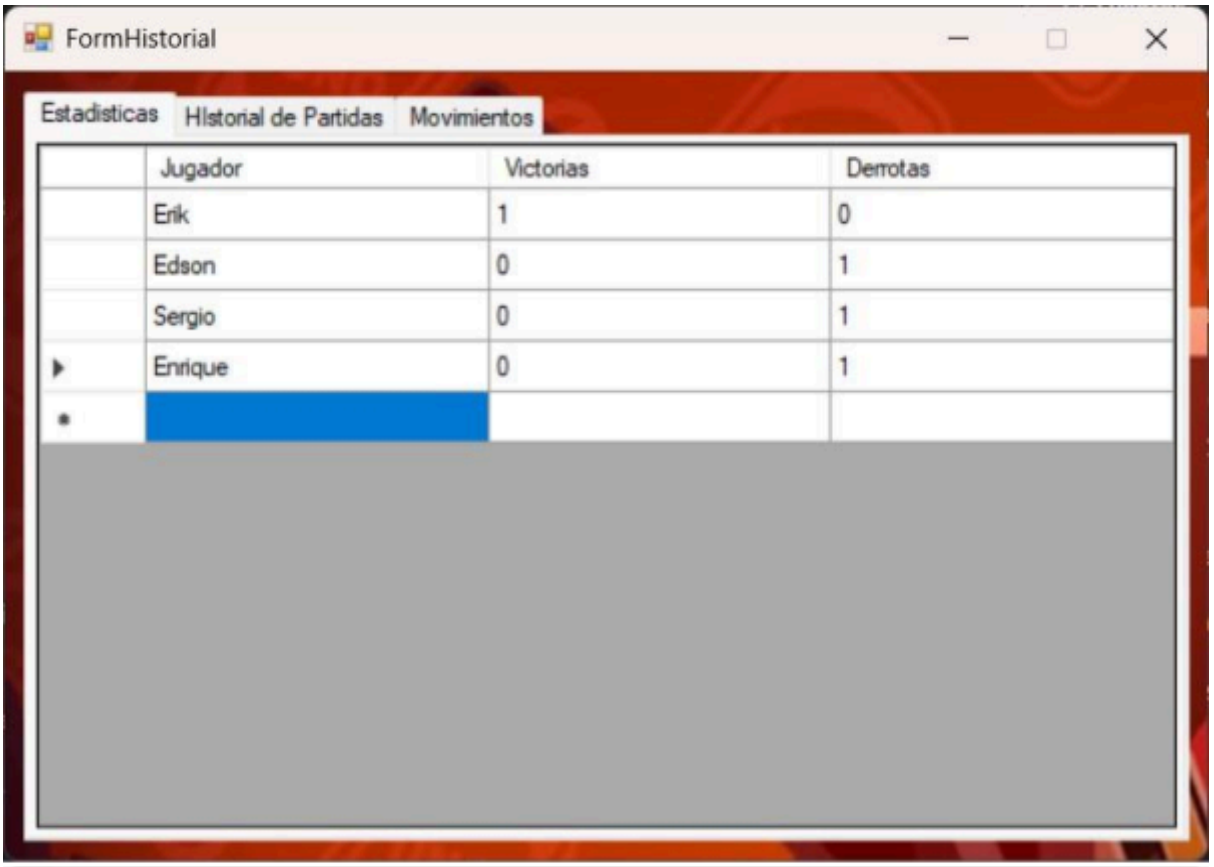

Juego UNO
Turno de: Edson → Siguiente turno es: Kike
Sonido:
Sentido: Izquierda
Erik: 3 cartas
Kike: 6 cartas
Sergio: 2 cartas
Color actual: Rojo
Edson: 4 cartas
Tu mano
Edson jugó Azul +2.
Kike come 2 cartas y pierde su turno.
Erik robó una carta y no era jugable, se la queda.
Sergio jugó Rojo +2.
Edson come 2 cartas y pierde su turno.
Kike jugó Rojo 9.
Erik jugó Rojo 2.
Sergio jugó Rojo 7.

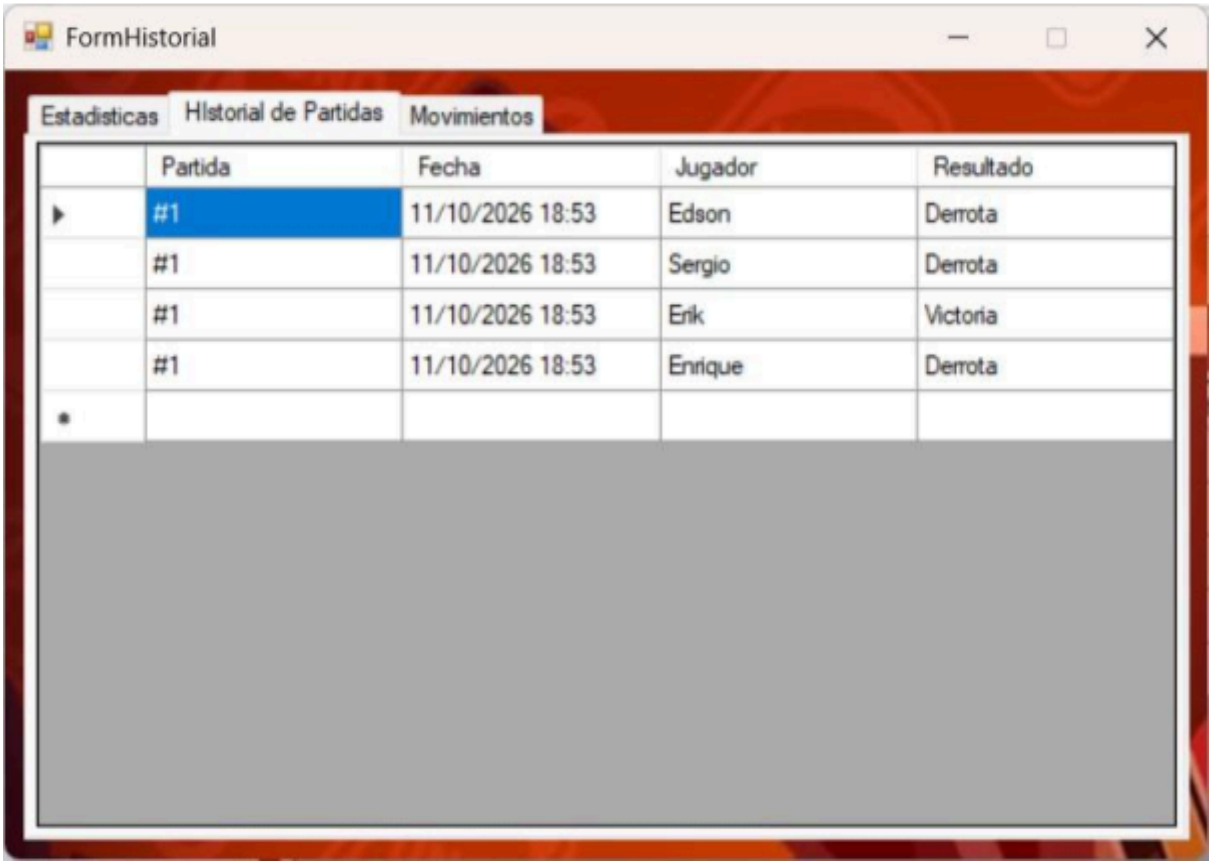

Juego UNO
Turno de: Sergio → Siguiente turno es: Edson
Sonido:
Sentido: Izquierda
Kike: 8 cartas
Edson: 3 cartas
Erik: 2 cartas
Color actual: Rojo
¡UNO!
Sergio: 2 cartas
Tu mano
jugable, se la queda.
Sergio jugó Rojo +2.
Edson come 2 cartas y pierde su turno.
Kike jugó Rojo 9.
Erik jugó Rojo 2.
Sergio jugó Rojo 7.
Edson jugó Rojo +2.
Kike come 2 cartas y pierde su turno.
Erik jugó Rojo 1.

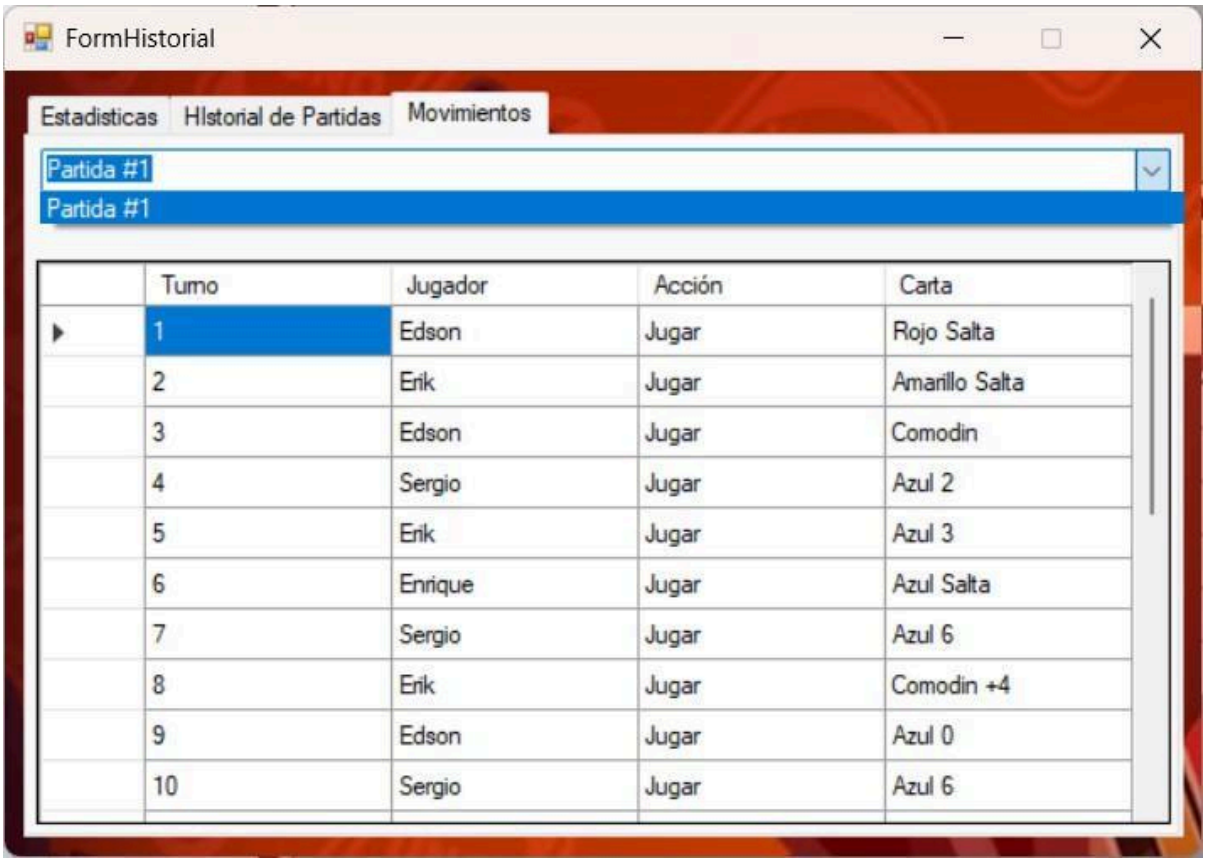

Sala de espera
Sala de espera
3/4
Edson (tú)
Sergio
Erik
Esperando jugador...
La partida empieza cuando se unan los 4 jugadores
Volver


Juego UNO
Sonido:
Turno de: Edson → Siguiente turno es: Enrique
Sentido: Derecha
Erik: 2 cartas
Enrique: 4 cartas
Sergio: 7 cartas
Edson: 8 cartas
Tu mano


Fin de la Partida
¡El ganador es: Erik!
UNO
Jugar de Nuevo
Volver al Menú


FormHistorial
Estadisticas
HIstorial de Partidas
Movimientos
Jugador
Victorias
Derrotas
Erik
1
0
Edson
0
1
Sergio
0
1
Enrique
0
1


FormHistorial
Estadisticas
HIstorial de Partidas
Movimientos
Partida
Fecha
Jugador
Resultado
#1
11/10/2026 18:53
Edson
Derrota
#1
11/10/2026 18:53
Sergio
Derrota
#1
11/10/2026 18:53
Erik
Victoria
#1
11/10/2026 18:53
Enrique
Derrota


FormHistorial
Estadisticas
HIstorial de Partidas
Movimientos
Partida #1
Partida #1
Turno
Jugador
Acción
Carta
1
Edson
Jugar
Rojo Salta
2
Erik
Jugar
Amarillo Salta
3
Edson
Jugar
Comodin
4
Sergio
Jugar
Azul 2
5
Erik
Jugar
Azul 3
6
Enrique
Jugar
Azul Salta
7
Sergio
Jugar
Azul 6
8
Erik
Jugar
Comodin +4
9
Edson
Jugar
Azul 0
10
Sergio
Jugar
Azul 6

PROBLEMAS ENCONTRADOS Y SOLUCIONES

Problema para subir cosas al repositorio: En un principio nadie sabía cómo hacer commits, por lo que tuvimos que ver un tutorial para hacerlo correctamente ya que si los hacíamos mal prácticamente arruinamos el proyecto ya que todo se subía al main.

En el commit de la preparación de las imágenes de las cartas el problema fue que esta rama se creó en un inicio, por lo que cuando se le daba en actualizar aparece un problema que no dejaba modificarlo, así que la solución más simple fue cambiarle el nombre poniendo como “v2”, de este modo ya se pudo trabajar y subirlo sin problemas.

Otro problema general que nos pasaba a todos es que cada que trabajabamos en un commit por alguna razón nos fallaba a nosotros, pero se hacían correcciones leves para que podamos seguir trabajando, esto con el tiempo ya no pasaba.

Un problema que tuvimos incluso al final del trabajo es que las animaciones de las cartas salían trabadas, arreglábamos algo y tronamos otra cosa, por lo que nos llevó bastante tiempo solucionarlo y aunque no quedó de la mejor manera, dimos nuestro mejor esfuerzo.

Un detalle a considerar era que como cada quien trabaja de forma diferente, en muchas ocasiones se llegaba a ver código que la otra persona no entendía, eso nos atrasaba las cosas para poder trabajar de forma correcta en el código, además de que al estar trabajando en issues secuenciales, nos teníamos que esperar al anterior para trabajar en el que sigue, igual esto hizo que empezara a sobrar código que no se usaba.

Otro detalle a considerar es que nosotros **empezamos el proyecto una semana y media después de que fue asignada**, por lo que los tiempos que tuvimos para desarrollar el trabajo fue menor, pero esto fue culpa **nuestra.**

Otro **problemota** es que en la segunda parte del proyecto solo tuvimos 3 días para hacer casi 20 commits pero otra vez fue **culpa nuestra**.

CONCLUSIONES POR INTEGRANTE

Erik: Fue un proyecto en el que desde que empezamos batalle, ya que no tenia conocimiento de como utilizar github, asi que atrase al equipo, pero ya después ya sabiendo ya agilice el trabajo y lo sacamos adelante; el proyecto en sí estuvo complicado pero me ayudó sobre todo a trabajar en equipo y organizar mejor las cosas, era inspirador ver cómo se iba construyendo el proyecto entre todos y saber que lo que uno hizo se iba adaptando hasta que salió este proyecto; con el aprendizaje en la vida profesional me va a servir para saber cómo lidiar bajo presión con otros compañeros, utilizar c# y python combinando estos lenguajes no fue sencillo, pero se logró.

Edson: En lo personal este fue un proyecto demasiado grande, diría que es el más grande que he hecho en todo lo que llevo de carrera, además de que este fue en equipo, algo que no se si complico o mejoro las cosas, fue dependiendo mucho de la situación, pero en lo personal fue un buen proyecto que siento que me puede ayudar en la vida profesional, más que nada por el tema de trabajar en equipo mediante github, ver como se usa la aplicación y que funciones trae. Algo que si me agrado fue el combinar dos lenguajes de programación diferentes como fue C# y Python, algo agradable también fue ver como github ayuda a que el trabajo en equipo sea más fácil, viendo como el proyecto se puede ir armando poco a poco o como ir subiendo partes pequeñas del proyecto y ver que cambio, en resumen me gustaría concluir que me gusto demasiado el proyecto por muchos ámbitos aunque también me hizo sufrir en muchos momentos.

Enrique: Fue un proyecto bastante interesante de hacer porque siempre fue fácil hacer el código individual, pero ahora teniendo varios colaboradores fue un problema entender el código y el razonamiento de los demás, también otro problema fue aprender a usar github pero después se logró trabajar correctamente en esa plataforma.
En la segunda parte donde se tenían que conectar varias pcs fue bastante complicado pero al final fue muy gratificante ver el juego terminado y como funciona a la perfección

Sergio: El proyecto te orilla mucho al trabajo en equipo, pero creo que resulta más fácil de organizar gracias a las herramientas como github, pues te permite el asignar tareas directamente y tener más visualmente como es el plan de trabajo y cómo poco a poco va creciendo el proyecto, pero como se dio el caso, hubo problemas usando esas herramientas, como una duplicación de código o inconsistencias en algún nombre de un evento, esto se tuvo que solucionar a mano para poder continuar. La implementación de la interfaz trajo consigo también varios desafíos y siendo un trabajo colaborativo también fue un tema a tratar el punto del diseño y mas que nada ponerse de acuerdo, pero al final creo que como equipo trabajamos de buena manera y al final logramos sacar un resultado aceptable.